



MIPI CSI-2 to PCIe Reference Design

Reference Design

FPGA-RD-02338-1.0

April 2026

Disclaimers

Lattice makes no warranty, representation, or guarantee regarding the accuracy of information contained in this document or the suitability of its products for any particular purpose. All information herein is provided AS IS, with all faults, and all associated risk is the responsibility entirely of the Buyer. The information provided herein is for informational purposes only and may contain technical inaccuracies or omissions, and may be otherwise rendered inaccurate for many reasons, and Lattice assumes no obligation to update or otherwise correct or revise this information. Products sold by Lattice have been subject to limited testing and it is the Buyer's responsibility to independently determine the suitability of any products and to test and verify the same. LATTICE PRODUCTS AND SERVICES ARE NOT DESIGNED, MANUFACTURED, OR TESTED FOR USE IN LIFE OR SAFETY CRITICAL SYSTEMS, HAZARDOUS ENVIRONMENTS, OR ANY OTHER ENVIRONMENTS REQUIRING FAIL-SAFE PERFORMANCE, INCLUDING ANY APPLICATION IN WHICH THE FAILURE OF THE PRODUCT OR SERVICE COULD LEAD TO DEATH, PERSONAL INJURY, SEVERE PROPERTY DAMAGE OR ENVIRONMENTAL HARM (COLLECTIVELY, "HIGH-RISK USES"). FURTHER, BUYER MUST TAKE PRUDENT STEPS TO PROTECT AGAINST PRODUCT AND SERVICE FAILURES, INCLUDING PROVIDING APPROPRIATE REDUNDANCIES, FAIL-SAFE FEATURES, AND/OR SHUT-DOWN MECHANISMS. LATTICE EXPRESSLY DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY OF FITNESS OF THE PRODUCTS OR SERVICES FOR HIGH-RISK USES. The information provided in this document is proprietary to Lattice Semiconductor, and Lattice reserves the right to make any changes to the information in this document or to any products at any time without notice.

Inclusive Language

This document was created consistent with Lattice Semiconductor's inclusive language policy. In some cases, the language in underlying tools and other items may not yet have been updated. Please refer to Lattice's inclusive language [FAQ 6878](#) for a cross reference of terms. Note in some cases such as register names and state names it has been necessary to continue to utilize older terminology for compatibility.

Contents

Contents	3
Acronyms and Abbreviations in This Document.....	6
1. Introduction	7
1.1. Quick Facts	7
1.2. Features	7
1.3. Naming Conventions	8
1.3.1. Nomenclature.....	8
1.3.2. Signal Names	8
2. Directory Structure and Files	9
3. Functional Description.....	10
3.1. Design Components	10
3.1.1. MIPI CSI/DSI RX IP.....	10
3.1.2. PCIe DMA	10
3.1.3. Debayer IP	10
3.1.4. Color Correction Matrix IP.....	11
3.1.5. Automatic White Balance IP.....	11
3.1.6. I2C Controller	11
3.1.7. Video to PCIe Bridge.....	11
3.2. Clocking Scheme	12
3.2.1. Clocking Overview	13
3.3. Reset Scheme	14
3.3.1. Reset Overview	14
4. Signal Description	15
5. Running the Reference Design	17
6. Implementing the Reference Design on Board	23
6.1. Hardware Setup	23
6.1.1. CertusPro-NX Versa Board RevB	23
6.1.2. IMX258 Sensor	24
6.1.3. Host PC	24
6.2. Programming the FPGA.....	24
6.2.1. Status LED.....	27
6.3. Software Setup.....	28
6.3.1. QV4L2 Application	28
6.3.2. lsc_pcie_video_bridge Driver	29
6.3.3. Running the QV4L2 Application and the lsc_pcie_video_bridge Driver.....	29
7. Customizing the Reference Design.....	31
7.1. Changing IP Configuration to 2 Lanes, 4 PPC	31
7.1.1. Configuring MIPI RX IP.....	31
7.2. Changing IP Configuration to 2 Lanes, 2 PPC	33
7.2.1. Configuring MIPI RX IP.....	33
7.2.2. Configuring Debayer IP.....	34
7.2.3. Configuring Color Correction Matrix IP	35
7.2.4. Configuring Automatic White Balance IP	37
7.2.5. Configuring Video to PCIe Bridge	38
8. Resource Utilization.....	40
9. Debugging.....	41
9.1. Error when Updating SPI Flash	41
9.2. Timing Violations.....	41
References.....	43
Technical Support Assistance	44
Revision History.....	45

Figures

Figure 2.1. Directory Structure	9
Figure 3.1. Reference Design Block Diagram	10
Figure 3.2. Reference Design Clock Domain Block Diagram	13
Figure 3.3. Reset Scheme Overview	14
Figure 5.1. Lattice Propel Builder Window	17
Figure 5.2. IP on Local Tab in IP Catalog	18
Figure 5.3. Install IP from File	18
Figure 5.4. IP License Agreement	19
Figure 5.5. Validate Design in Lattice Propel Builder	19
Figure 5.6. TCL Console Output after Validating Design	19
Figure 5.7. Generate in Lattice Propel Design	19
Figure 5.8. TCL Console Output after Generating Design	19
Figure 5.9. Lattice Radiant Icon in Lattice Propel Builder	20
Figure 5.10. Lattice Radiant Window	20
Figure 5.11. Synthesis Strategy Settings	21
Figure 5.12. Place and Route Design Strategy Settings	21
Figure 5.13. Generating Bitstream	22
Figure 5.14. Compilation Completed	22
Figure 6.1. MIPI CSI-2 to PCIe Hardware Setup	23
Figure 6.2. CertusPro-NX Versa Board	23
Figure 6.3. IMX258 Sensor Module	24
Figure 6.4. Sample PCIe Slot that Supports Up to PCIe x16	24
Figure 6.5. Creating a New Project from a Scan	25
Figure 6.6. Lattice Radiant Programmer Window	25
Figure 6.7. CertusPro-NX Device Settings	25
Figure 6.8. Device Properties Window for CertusPro-NX SPI Flash Programming	26
Figure 6.9. Programming Menu Bar	26
Figure 6.10. Programmer Output Window	27
Figure 6.11. CertusPro-NX Status LEDs	27
Figure 6.12. QV4L2 Application Window	28
Figure 6.13. MIPI CSI-2 to PCIe Bridge Output Image Using the QV4L2 Application	30
Figure 7.1. MIPI RX IP Configuration	32
Figure 7.2. MIPI RX IP Check Generated Result Window	32
Figure 7.3. MIPI RX IP Configuration	33
Figure 7.4. MIPI RX IP Check Generated Result Window	34
Figure 7.5. Debayer IP Configuration	34
Figure 7.6. Debayer IP Check Generated Result Window	35
Figure 7.7. Color Correction Matrix IP Configuration	36
Figure 7.8. Color Correction Matrix IP Check Generated Result Window	36
Figure 7.9. Automatic White Balance IP Configuration	37
Figure 7.10. Automatic White Balance IP Check Generated Result Window	38
Figure 7.11. Video to PCIe Bridge Configuration	38
Figure 7.12. Video to PCIe Bridge Check Generated Result Window	39
Figure 9.1. TCK Frequency Settings	41
Figure 9.2. Port Selection	41
Figure 9.3. Place & Route Design Strategy Settings	42

Tables

Table 1.1. Summary of the Reference Design7

Table 2.1. File List9

Table 3.1. Video to PCIe Bridge Parameters11

Table 3.2. Video to PCIe Bridge Signals12

Table 4.1. Primary I/O15

Table 6.1. Description of the Status LED.....27

Table 6.2. Configurable Settings in the QV4L2 Application28

Table 7.1. IP Configuration for Supported Resolutions31

Table 8.1. Resource Utilization of the CertusPro-NX MIPI CSI-2 to PCIe Reference Design40

Acronyms and Abbreviations in This Document

A list of acronyms and abbreviations used in this document.

Acronyms/Abbreviations	Definition
AWB	Automatic White Balance
AXI4-S	Advanced eXtensible Interface 4 – Stream
AXI4-Lite	AMBA AXI4-Lite
B2P	Byte-to-Pixel
BGR3	V4L2 pixel format
BPC	Bits Per Component
BPP	Bits Per Pixel
C	C Programming Language
C++	C++ Programming Language
CCI	Camera Control Interface
CCM	Color Correction Matrix
CSI-2	Camera Serial Interface-2
D-PHY	D-PHY physical layer
DCFIFO	Dual-Clock FIFO
DMA	Direct Memory Access
DSP	Digital Signal Processor
EBR	Embedded Block RAM
FPGA	Field-Programmable Gate Array
GUI	Graphical User Interface
HDL	Hardware Description Language
I2C	Inter-Integrated Circuit
IP	Intellectual Property
ISP	Image Signal Processor
LED	Light-Emitting Diode
LTS	Long-Term Support
LUT4	4-input Look-Up Table
Mbps	Megabits Per Second
MIPI	Mobile Industry Processor Interface
OS	Operating System
PCIe	Peripheral Component Interconnect Express
PLL	Phase-Locked Loop
PMA	Physical Medium Attachment
PPC	Pixels Per Clock
QV4L2	Qt Video4Linux2 viewer
RAW10	10-bit Bayer RAW format
RGB	Red, Green, Blue
RTL	Register Transfer Level
RX	Receiver
SCL	Serial Clock Line
SDA	Serial Data Line
SPI	Serial Peripheral Interface
TCL	Tool Command Language
TX	Transmitter
UVSI	Unified Video Streaming Interface
V4L2	Video4Linux2

1. Introduction

The CertusPro™-NX Mobile Industry Processor Interface (MIPI®) Camera Serial Interface-2 (CSI-2) to Peripheral Component Interconnect Express (PCIe®) reference design demonstrates the functionality of transferring MIPI CSI-2 sensor video data to a computer through PCIe with a direct memory access (DMA) engine. Built on the CertusPro-NX Versa Board, this design includes Linux® operating system (OS) driver support. This reference design showcases the complete data path—from capturing sensor data, transferring the data into the computer memory via PCIe and DMA, to rendering the video on the computer screen through the provided software driver.

1.1. Quick Facts

Download the [MIPI CSI-2 to PCIe Reference Design](#) design files from GitHub.

Table 1.1. Summary of the Reference Design

General	Target Devices	CertusPro-NX
	Source code format	Verilog RTL, C, C++
Simulation	Functional simulation	Not performed
	Timing simulation	Not performed
	Test bench	Not available
	Test bench format	Not applicable
Software Requirements	Software tool and version	• Lattice™ Propel™ Builder 2025.2 • Lattice Radiant™ Software 2025.2 • Lattice Radiant Programmer 2025.2 • Ubuntu® 22.04.5 LTS 64-bit • Linux Driver source code • Linux V4L2 application
	IP version (if applicable)	• MIPI CSI/DSI RX IP version 1.0.0.01 (Custom IP) • PCIe x4 IP version 4.0.0 • Debayer IP version 1.4.0.01 (Custom IP) • Color Correction Matrix IP version 1.3.1.01 (Custom IP) • Automatic White Balance IP version 1.4.0.01 (Custom IP)
Hardware Requirements	Board	CertusPro-NX Versa Board RevB
	Cable	Mini USB Type A cable
	Others	• Sony® IMX258 sensor module • Computer with Linux OS

1.2. Features

Key features of the MIPI CSI-2 to PCIe reference design include:

- The Lattice MIPI CSI/DSI RX IP in this reference design is configured to support two or four lanes with lane rate of 1200 Mbps, as capped by IMX258 capability. The IP receives, processes, and converts incoming MIPI video payload packets up to 4K30 into Unified Video Streaming Interface (UVSI) packets. For details, refer to the <Project Directory>/latticesemi_ip_mipi_csi_dsi_1.0.0.01/doc/IP_Design_Notes.pdf.
- The Lattice PCIe x4 IP provides DMA engine to perform data transfer between FPGA and host PC through PCIe Gen3x4 link. The IP is configured in DMA Bridge mode, providing an AXI4 interface that enables camera control from the host PC.
- The Lattice Debayer IP extracts the R, G, and B components from the pixel data output by the image sensor, converting the RAW10 data format into full RGB components. For details, refer to the < Project Directory>/latticesemi_custom_ip_debayer_1.4.0.01/doc/IP_Design_Notes.pdf.

- The Lattice Color Correction Matrix IP performs pixel data correction by adjusting R, G, and B components gain and compensates for color channel crosstalk. For details, refer to the < *Project Directory*>/latticesemi_custom_ip_ccm_1.3.1.01/doc/IP_Design_Notes.pdf.
- The Lattice Automatic White Balance IP automatically compensates for illumination temperature-based color differences in Bayer domain, such that white appears white. For details, refer to the < *Project Directory*>/latticesemi_custom_ip_awb_1.4.0.01/doc/IP_Design_Notes.pdf.
- The Unified Video to PCIe bridge maps UVSI packets to PCIe DMA Advanced eXtensible Interface-Stream (AXI4-S) packets.

1.3. Naming Conventions

1.3.1. Nomenclature

The nomenclature used in this document is based on Verilog HDL.

1.3.2. Signal Names

- `_n` are active low signals (asserted when value is logic 0)
- `_i` are input signals
- `_o` are output signals
- `_io` are bi-directional input and output signals

2. Directory Structure and Files

Figure 2.1 shows the directory structure.

```

+---docs //Contains reference design user guide
+---fpga_lfcpnx //FPGA_DEVICE_FAMILY
|   +---precompiled_file //Contains a precompiled bitstream (IP Evaluation) for the default configuration
|   \---radiant //Contains Lattice Propel Builder files, Lattice Radiant software files, and RTL source code
|       +---propelbld_mipi_csi_to_pcie
|       |   \---lib
|       |       +---IPs
|       |       |   \---latticesemi.com
|       |       |       +---ip
|       |       |       |   +---axi_to_apb
|       |       |       |   +---i2c_ctrl
|       |       |       |   +---mipi_csi_rx
|       |       |       |   +---pcie_axis_dma
|       |       |       |   \---module
|       |       |       |       +---sys_clk_pll
|       |       |       |       \---latticesemi_custom
|       |       |       |           +---ip
|       |       |       |           |   +---isp_awb
|       |       |       |           |   +---isp_ccm
|       |       |       |           |   +---isp_debayer
|       |       |       |           |   +---video_pcie_bridge
|       |       |       |           \---sge //System Generation Environment (Generated by Lattice Propel Builder, this is an empty folder for the reference design)
|       |       |       |
|       |       |       +---src
|       |       |       |   \---constraints //Contains the constraints file (.pdc) used in the design
|       |       |       |   \---verification //Generated by the Lattice Propel Builder
|       |       |       \---misc //Contains non-FPGA files
|       |       |           +---custom_ip //Contains Lattice IP Packagers for custom IPs (For example, MIPI CSI/DSI RX and Debayer)
|       |       |           \---software //Contains instructions to download the software required
|       |       \---sge //System Generation Environment (Generated by Lattice Propel Builder, this is an empty folder for the reference design)
|       \---src
|           \---constraints //Contains the constraints file (.pdc) used in the design
|           \---verification //Generated by the Lattice Propel Builder
+---misc //Contains non-FPGA files
|   +---custom_ip //Contains Lattice IP Packagers for custom IPs (For example, MIPI CSI/DSI RX and Debayer)
|   \---software //Contains instructions to download the software required

```

Figure 2.1. Directory Structure

Table 2.1 shows the list of files included in the reference design package.

Table 2.1. File List

Attribute	Description
<Component name>.ipx	Contains the information on the files associated with the generated IP.
<Component name>.cfg	Contains the parameter values used in IP configuration.
component.xml	Contains the ipxact:component information of the IP.
design.xml	Documents the configuration parameters of the IP in IP-XACT 2014 format.
rtl/<Component name>.v	Provides an example RTL top file that instantiates the module.
rtl/<Component name>_bb.v	Provides the synthesis closed-box.
misc/<Component name>_tmpl.v	Provides instance templates for the module.
misc /<Component name>_tmpl.vhd	

3. Functional Description

The MIPI CSI-2 to PCIe reference design captures video data from an external camera and processes the data through the CertusPro-NX device before transmitting the processed data to a PCIe host. The MIPI CSI/DSI RX with Byte-to-Pixel (B2P) conversion block accepts raw MIPI CSI-2 data streams from the camera and converts the data into a suitable pixel format. The ISP pipeline then performs essential image processing functions, including Debayer to reconstruct full-color pixels, apply a color correction matrix (CCM) for accurate color reproduction, and execute automatic white balance (AWB) adjustments. The Unified Video-to-Pcie block packages the processed video frames into PCIe-compatible data structures. Finally, the PCIe interface transfers the formatted video data to the PCIe host system for memory storage and display.

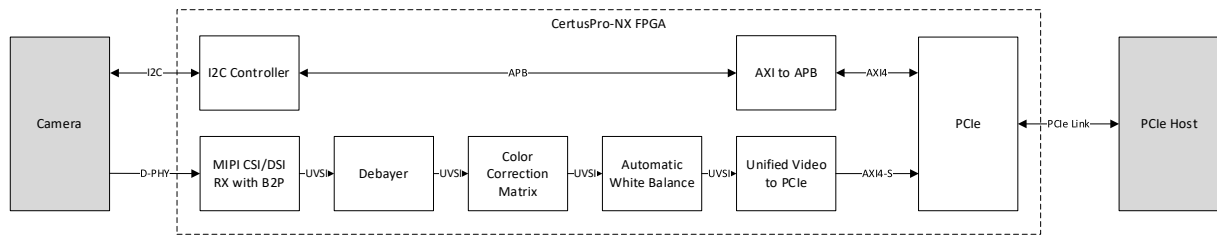


Figure 3.1. Reference Design Block Diagram

3.1. Design Components

The MIPI CSI-2 to PCIe reference design includes blocks described in the following sections.

3.1.1. MIPI CSI/DSI RX IP

The Lattice MIPI CSI/DSI D-PHY IP provides a flexible solution for receiving high-speed video data over a MIPI-compliant D-PHY interface in FPGA designs. MIPI enables high-speed, low power video transmission between components such as cameras and displays. In this reference design, the video packets are received through D-PHY physical layer interface from the IMX258 sensor. The incoming video payload packets are processed and converted from bytes to pixels before transmitted through UVSI. For complete specifications, including interfaces signal and parameters details, refer to the MIPI CSI/DSI RX IP documentation: < Project

Directory>/latticesemi.com_ip_mipi_csi_dsi_1.0.0.01/doc/IP_Design_Notes.pdf.

3.1.2. PCIe DMA

The DMA engine provides a high-performance DMA data transmission between FPGA and host PC using the driver software through the PCIe Gen3x4 link. The PCIe DMA with Bridge mode provides an AXI4 interface for the host PC to read and write I2C controller registers for camera control. For more information, refer to the [Software Setup](#) section and the [PCI Express for Nexus FPGAs](#) web page.

3.1.3. Debayer IP

The image sensor outputs pixel data in Bayer format. To extract the individual R, G, and B color components from this Bayer-patterned data, a Debayer function is required. The Debayer IP converts RAW10 image data into full RGB components. This IP supports UVSI interfaces for both input and output video streams. For complete specifications, including interfaces signal and parameters details, refer to the Debayer IP documentation: < Project

Directory>/latticesemi_custom_ip_debayer_1.4.0.01/doc/IP_Design_Notes.pdf.

3.1.4. Color Correction Matrix IP

To obtain the correct colors, the IP performs pixel data correction by adjusting R, G, and B components gain and compensates for color channel crosstalk. This linear mapping of the color components is achieved by using a 3x3 matrix, called color correction matrix (CCM). For complete specifications, including interfaces signal and parameters details, refer to the Color Correction Matrix IP documentation: < Project Directory>/latticesemi_custom_ip_ccm_1.3.1.01/doc/IP_Design_Notes.pdf.

3.1.5. Automatic White Balance IP

The Automatic White Balance (AWB) IP automatically corrects illumination-dependent color shifts to ensure accurate white reproduction. The IP operates primarily in the Bayer domain but also supports RGB processing. The AWB algorithm uses scene-based image statistics, requires no calibration, and performs two main steps: detects the illuminant and applies pixel-level correction through gain adjustment. The IP supports UVSI interfaces, processes up to 16 bits per pixel and 4 pixels per clock, and offers an optional AXI4-Lite interface for dynamic configuration and status monitoring. For complete specifications, including interfaces signal and parameters details, refer to the AWB IP documentation: < Project Directory>/latticesemi_custom_ip_awb_1.4.0.01/doc/IP_Design_Notes.pdf.

3.1.6. I2C Controller

In this reference design, an I2C controller is used exclusively for controlling the camera device. The controller receives read and write commands requested from the host PC through the PCIe bridge, then executes the I2C transaction through the camera control interface (CCI) to configure the camera internal registers and initiate the camera streaming process.

3.1.7. Video to PCIe Bridge

The Video to PCIe bridge receives video data from UVSI, packs the data into a 256-bit data bus, buffers and safely transfers the data from the pixel clock domain to the PCIe user clock domain using a standard DCFIFO, and manages the AXI4-S handshake between this module output and the PCIe DMA input. In addition, this module sends an interrupt per frame to the PCIe IP to indicate to the host PC that a video frame transfer is complete. The interrupt request and acknowledgement are handled in this module.

This module enables seamless data flow from UVSI to the PCIe IP while minimizing the number of design blocks you must instantiate to perform the transfer.

Table 3.1. Video to PCIe Bridge Parameters

Port Name	Selectable Values	Default	Description
BPC	6, 8, 10, 12, 16	8	Bits per component.
ACTIVE_BPC	6, 8, 10, 12, 16	8	Active bits per component. When the active bits per component are less than the configured bits per component, only the upper bits of the color component are processed by the Video to PCIe bridge.
CPP	1,2,3	3	Component per pixel
ACTIVE_CPP	1,2,3	3	Active component per pixel. When the active component per pixel is less than the configured component per pixel, only the lower components of the pixel are processed by the Video to PCIe bridge.
PPC	1,2,4	4	Pixel per clock.
TDATA_WIDTH	—	—	The width of the AXI-S UVSI data bus which is equal to the next higher multiple of 8 based on the quantity of BPC, CPP, and PPC. Not user editable. Calculated based on BPC, CPP, and PPC in accordance to the UVSI specification.

Table 3.2. Video to PCIe Bridge Signals

Port Name	I/O Type	Width	Description
axis_vid_clk_i	Input	1	UVSI pixel clock.
pcie_dma_clk_i	Input	1	PCIe DMA user clock.
rst_n	Input	1	Active low reset.
start_streaming	Output	1	Video streaming start indicator.
Unified Video Streaming Interface			
axis_vid_tvalid_i	Input	1	TVALID indicates the availability of valid pixel data on axis_vid_tdata_i when TREADY is asserted.
axis_vid_tdata_i	Input	[TDATA_WIDTH-1:0]	TDATA is the primary payload signal used to transfer data across the interface.
axis_vid_tuser_i	Input	[1:0]	Sideband information transmitted alongside the data stream. axis_vid_tuser_i[0] – Start of frame axis_vid_tuser_i[1] – Interlaced video
axis_vid_tlast_i	Input	1	TLAST indicates the end of each horizontal line and always coincides with the last pixel of the line.
axis_vid_tready_o	Output	1	TREADY indicated that RX can accept a transfer in the current cycle.
PCIe DMA AXI4-S Interface			
pcie_dma_tvalid_o	Output	1	TVALID indicates availability of valid pixel data on pcie_dma_tdata_o when TREADY is asserted.
pcie_dma_tdata_o	Output	[255:0]	TDATA is the primary payload signal used to transfer data across the interface.
pcie_dma_tlast_o	Output	1	TLAST indicates the last data of a frame.
pcie_dma_tready_i	Input	1	TREADY indicated that PCIe can accept a transfer in the current cycle
Interrupt			
pcie_dma_int_req_o	Output	1	PCIe DMA interrupt request.
pcie_dma_int_ack_i	Input	1	PCIe DMA interrupt acknowledge.

3.2. Clocking Scheme

An overview of the MIPI CSI-2 to PCIe reference design clocking scheme is shown in [Figure 3.2](#). In this reference design, a PLL is required to generate 250 MHz, 125 MHz, 150 MHz, and 100 MHz frequencies to specific modules. These generated clocks are routed to the corresponding submodules for further processing.

3.2.1. Clocking Overview

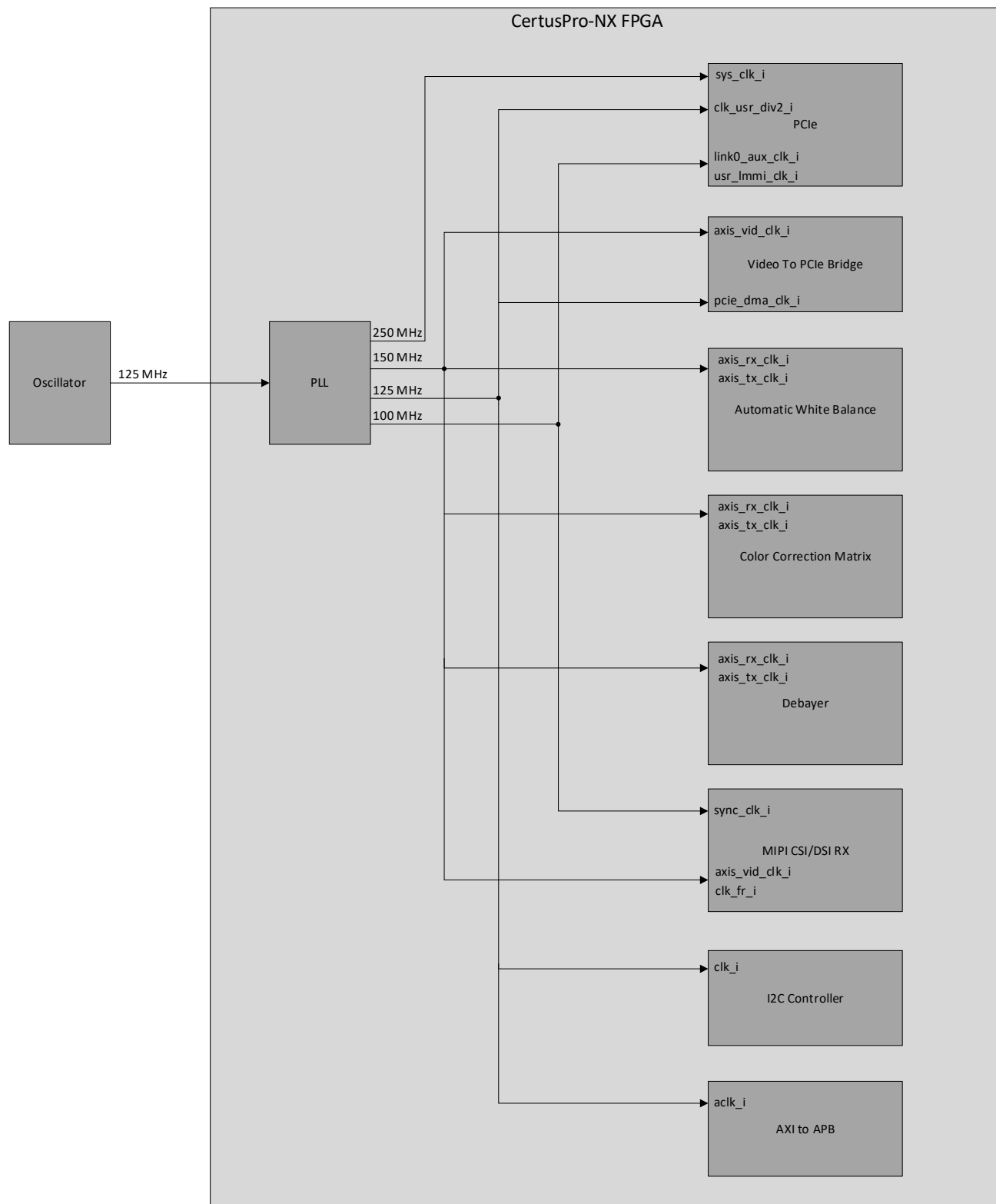


Figure 3.2. Reference Design Clock Domain Block Diagram

3.3. Reset Scheme

This reference design does not require the use of synchronized resets.

3.3.1. Reset Overview

An overview of the MIPI CSI-2 to PCIe reference design reset scheme is shown in [Figure 3.3](#).

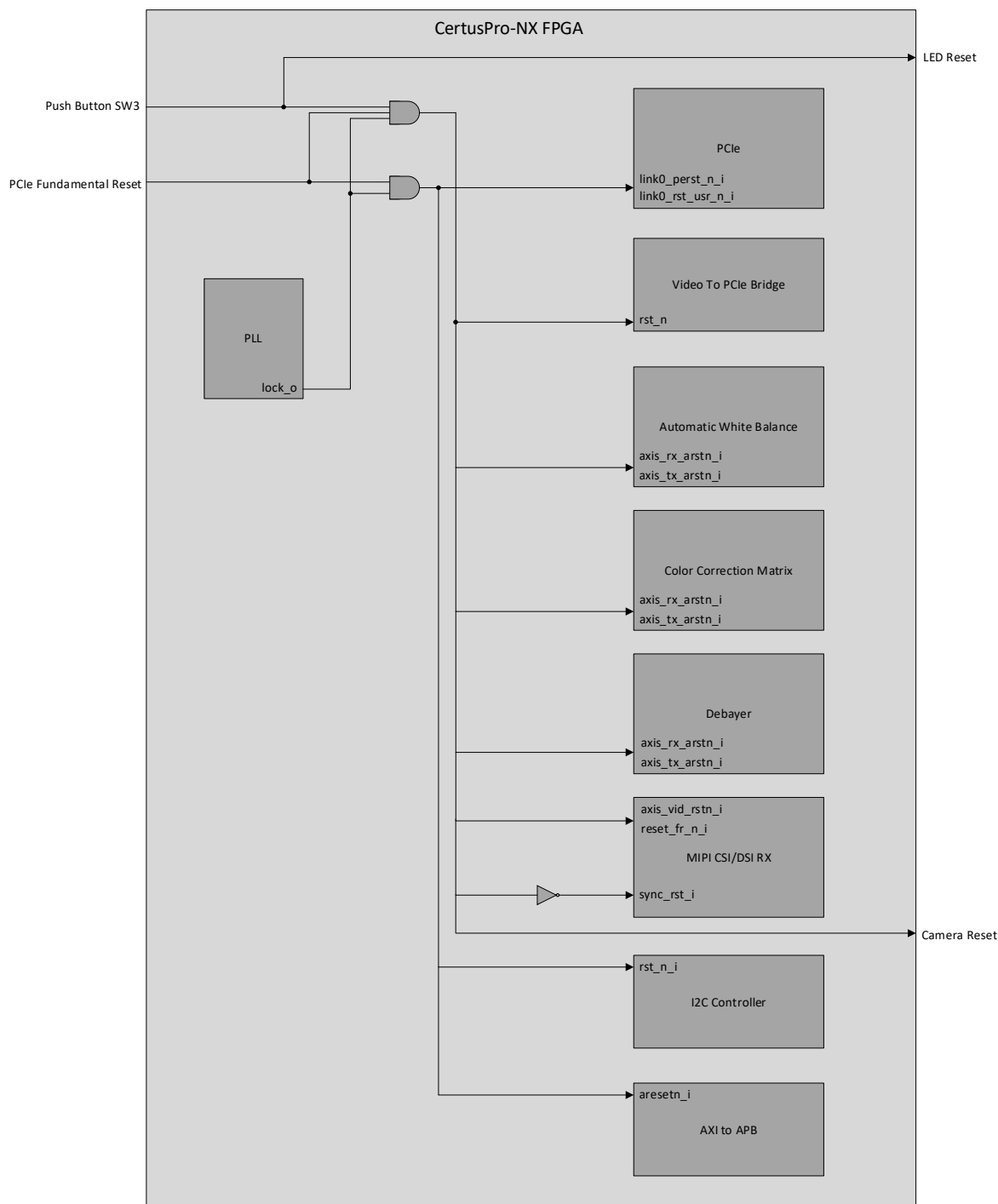


Figure 3.3. Reset Scheme Overview

4. Signal Description

The table below shows the input and output interface signals for this reference design.

Table 4.1. Primary I/O

Port Name	I/O Type	Width	Description
clk_125mhz	Input	1	Reference clock input for PLL.
pcie_perst_n_i	Input	1	Active low reset input for PCIe fundamental reset.
rst_n	Input	1	Active low reset input for design. Activates by pressing push button SW3 on the board.
pcie_sd0_rxdp_i	Input	1	Bit [0] of differential receive serial signal, RX+.
pcie_sd1_rxdp_i	Input	1	Bit [1] of differential receive serial signal, RX+.
pcie_sd2_rxdp_i	Input	1	Bit [2] of differential receive serial signal, RX+.
pcie_sd3_rxdp_i	Input	1	Bit [3] of differential receive serial signal, RX+.
pcie_sd0_rxdn_i	Input	1	Bit [0] of differential receive serial signal, RX-.
pcie_sd1_rxdn_i	Input	1	Bit [1] of differential receive serial signal, RX-.
pcie_sd2_rxdn_i	Input	1	Bit [2] of differential receive serial signal, RX-.
pcie_sd3_rxdn_i	Input	1	Bit [3] of differential receive serial signal, RX-.
pcie_sd0_refret_i	Input	1	Bit [0] of analog reference return for PMA PLL.
pcie_sd1_refret_i	Input	1	Bit [1] of analog reference return for PMA PLL.
pcie_sd1_refret_i	Input	1	Bit [2] of analog reference return for PMA PLL.
pcie_sd1_refret_i	Input	1	Bit [3] of analog reference return for PMA PLL.
pcie_sd0_rext_i	Input	1	Bit [0] of external resistance.
pcie_sd1_rext_i	Input	1	Bit [1] of external resistance.
pcie_sd2_rext_i	Input	1	Bit [2] of external resistance.
pcie_sd3_rext_i	Input	1	Bit [3] of external resistance.
pcie_refclkp_i	Input	1	Differential reference clock, CLK+. Default is 100 MHz, 125 MHz is also available.
pcie_refclkn_i	Input	1	Differential reference clock, CLK-. Default is 100 MHz, 125 MHz is also available.
pcie_status_led	Output	5	LED to indicate the status of system signals, including PLL lock signal, PCIe physical layer linkup, data link layer linkup, transaction layer linkup, and DMA start.
pcie_sd0_txdp_o	Output	1	Bit [0] of differential transmit serial signal, TX+.
pcie_sd1_txdp_o	Output	1	Bit [1] of differential transmit serial signal, TX+.
pcie_sd2_txdp_o	Output	1	Bit [2] of differential transmit serial signal, TX+.
pcie_sd3_txdp_o	Output	1	Bit [3] of differential transmit serial signal, TX+.
pcie_sd0_txdn_o	Output	1	Bit [0] of differential transmit serial signal, TX-.
pcie_sd1_txdn_o	Output	1	Bit [1] of differential transmit serial signal, TX-.
pcie_sd2_txdn_o	Output	1	Bit [2] of differential transmit serial signal, TX-.
pcie_sd3_txdn_o	Output	1	Bit [3] of differential transmit serial signal, TX-.
cpnx_versa_pcie_sw_sel_o	Output	1	A pin to select PCIe operating mode.
cpnx_versa_pcie_sw1_pd_o	Output	1	Control pin 1 that manages the power-down state of the board first PCIe switching element.
cpnx_versa_pcie_sw2_pd_o	Output	1	Control pin 2 that manages the power-down state of the board first PCIe switching element
cpnx_versa_pcie_clk_sel_o	Output	1	PCIe reference-clock selection pin on the CertusPro-NX Versa Board.
cam_reset_n_o	Output	1	Active low reset control pin for the external MIPI CSI-2 camera module.
rst_led	Output	1	LED to indicate reset signal for the PLL reference clock

Port Name	I/O Type	Width	Description
			domain.
cam_scl_io	Inout	1	I2C controller serial clock line (SCL) used for communication with MIPI CSI-2 camera module.
cam_sda_io	Inout	1	I2C controller serial data line (SDA) used for communication with MIPI CSI-2 camera module.
d_n_io	Inout	[NUMLANE ¹ -1:0]	Negative differential data I/O pad for D-PHY implementations.
d_p_io	Inout	[NUMLANE ¹ -1:0]	Positive differential data I/O pad for D-PHY implementations.
clk_n_io	Inout	1	Negative differential clock I/O pad for D-PHY implementations.
clk_p_io	Inout	1	Positive differential clock I/O pad for D-PHY implementations.

Note:

1. Corresponds to the *Number of D-PHY RX Lanes* attribute.

To run the reference design in the Lattice Propel Builder and Lattice Radiant software, follow these steps:

-

3. To reconfigure the IP, follow these steps:
 - a. Double-click the block diagram.
 - b. Modify the desired parameters in the configuration GUI.
 - c. Click **Generate** and **Finish** to apply the changes.
4. Before reconfiguring the IP, ensure that the IP is downloaded and installed under **IP on Local** as shown in [Figure 5.2](#). In this reference design, you need to install a few IPKs into **IP on Local**. To install the IPKs, follow these steps:
 - a. Right-click **IP on Local** and select **Install IP from file...**, as shown in [Figure 5.3](#).
 - b. In the pop-up file browser, select the following IPK path:
`<Project Directory>\RD02338 mipi csi to pcie\misc\custom ip\latticesemi.com mipi csi dsi 1.0.0.01.ipk`

- c. Click **OK**.
- d. In the **IP License Agreement** window, click **Accept** to finish the IP installation, as shown in [Figure 5.4](#).
- e. Repeat step 4a to 4d to install the following IPKs:
 - <Project Directory>\RD02338_mipi_csi_to_pcie\misc\custom_ip\latticesemi_custom_awb_1.4.0.01.ipk
 - <Project Directory>\RD02338_mipi_csi_to_pcie\misc\custom_ip\latticesemi_custom_ccm_1.3.1.01.ipk
 - <Project Directory>\RD02338_mipi_csi_to_pcie\misc\custom_ip\latticesemi_custom_debayer_1.4.0.01.ipk
 - <Project Directory>\RD02338_mipi_csi_to_pcie\misc\custom_ip\latticesemi_custom_unified_video_to_pcie_bridge_1.1.0.ipk

Note: Refer to the [Customizing the Reference Design](#) section for the available IP configuration options.

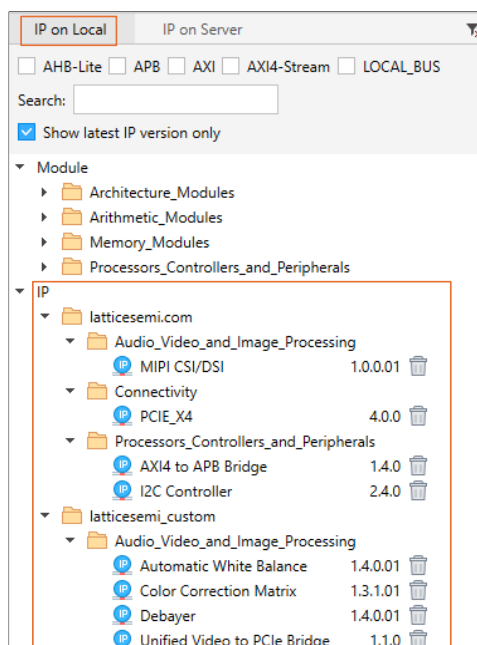


Figure 5.2. IP on Local Tab in IP Catalog

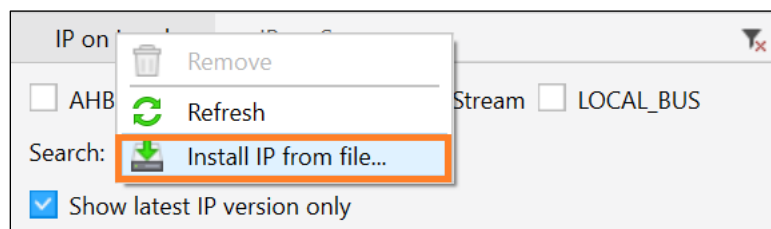


Figure 5.3. Install IP from File

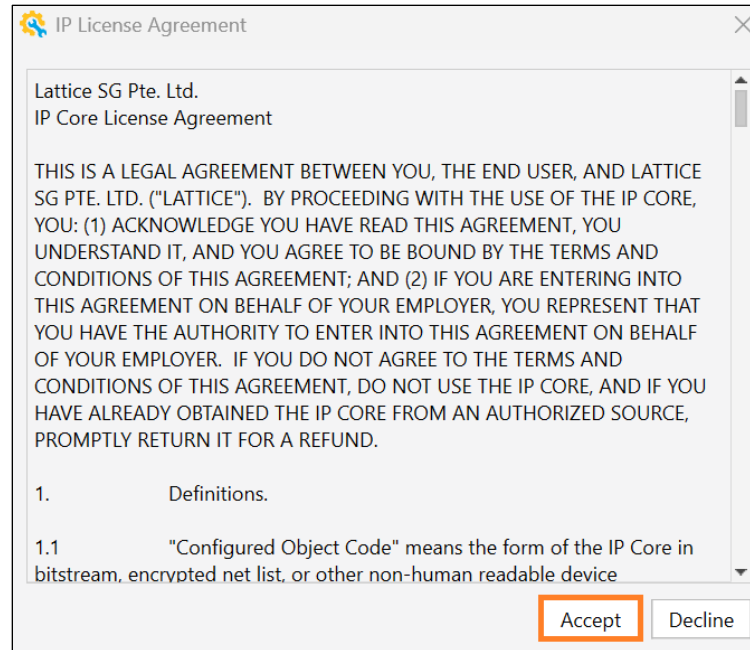


Figure 5.4. IP License Agreement

- To validate the design, go to **Design > Validate Design** or click the **Validate Design** icon (as shown in Figure 5.5). Ensure the TCL console shows no error, and only the expected INFO messages and WARNINGS appear.

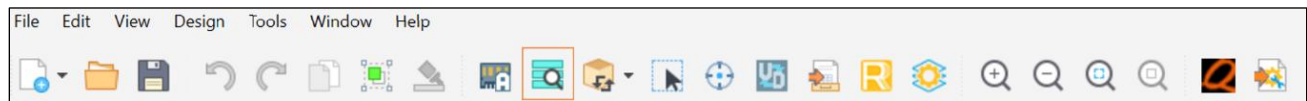


Figure 5.5. Validate Design in Lattice Propel Builder

```
$ sbp_design drc
INFO <2359136> - Start: sbp_design drc.
INFO <2359992> - Dangling inputs are set to default value (ARREGION=0,AWQOS=0,AWREGION=0,AWUSER=0,WUSER=0) in axi_to_apb_inst.AXI4_S
WARNING <2355109> - Cannot find memory map for component 'video_pcie_bridge_inst' for interface 'AXI4S_VID'.
WARNING <2357005> - Component 'video_pcie_bridge_inst' does not contain a memory map for interface 'AXI4S_VID'.
INFO <2359137> - Finished successfully: sbp_design drc.
$
```

Figure 5.6. TCL Console Output after Validating Design

- To generate the design, go to **Design > Generate** or click the **Generate** icon (as shown in Figure 5.7). Ensure the TCL console shows no errors, and only the expected INFO messages and WARNINGS appear.

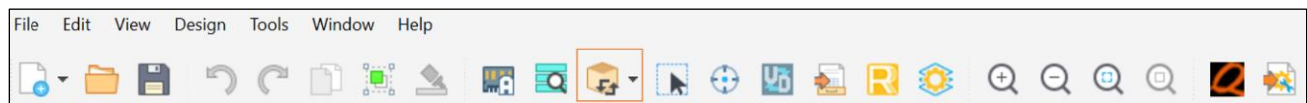


Figure 5.7. Generate in Lattice Propel Design

```
$ sbp_design generate
INFO <2359189> - Start: sbp_design generate.
INFO <2359992> - Dangling inputs are set to default value (ARREGION=0,AWQOS=0,AWREGION=0,AWUSER=0,WUSER=0) in axi_to_apb_inst.AXI4_S
WARNING <2355109> - Cannot find memory map for component 'video_pcie_bridge_inst' for interface 'AXI4S_VID'.
WARNING <2357005> - Component 'video_pcie_bridge_inst' does not contain a memory map for interface 'AXI4S_VID'.
INFO <2359190> - Finished successfully: sbp_design generate.
$ sbp_design save
$ sbp_design pge sge -i (C:/Work/mipi_csi_to_pcie/fpga_cpnx/radiant/propelbld_mipi_csi_to_pcie/mipi_csi_to_pcie.sbx) -o (C:/Work/mipi_csi_to_pcie/fpga_cpnx/radiant/propelbld_mipi_csi_to_pcie/...) -nc
WARNING <2359991> - (PGE SoCDesign) i2c_ctrl_inst_INTR remains unconnected
WARNING <2359991> - (PGE) No cpu instance, so no soc_svd generation, no sys_env.xml for SDK
$
```

Figure 5.8. TCL Console Output after Generating Design

- To synthesize the RTL files and generate the bitstream, click the Lattice Radiant icon in the Lattice Propel Builder as shown in [Figure 5.9](#).

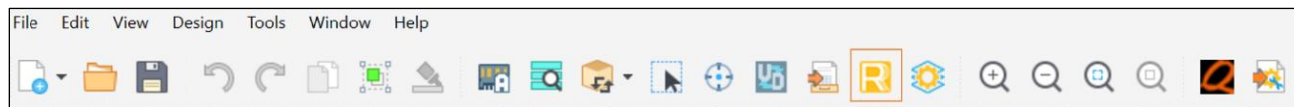


Figure 5.9. Lattice Radiant Icon in Lattice Propel Builder

- The Lattice Radiant software window of this project opens, as shown in [Figure 5.10](#).

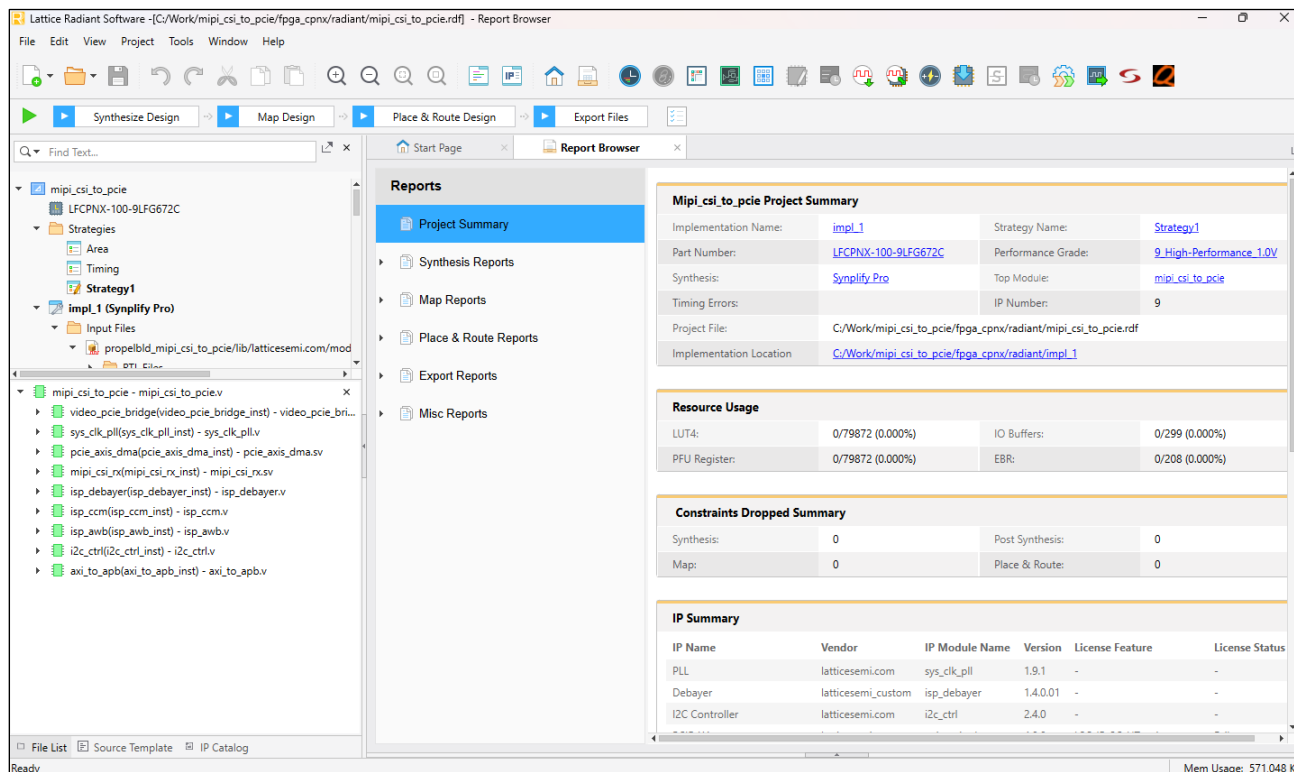


Figure 5.10. Lattice Radiant Window

9. Set for the Synthesis strategy as shown in Figure 5.11 and the Place and Route design strategy as shown in Figure 5.12.

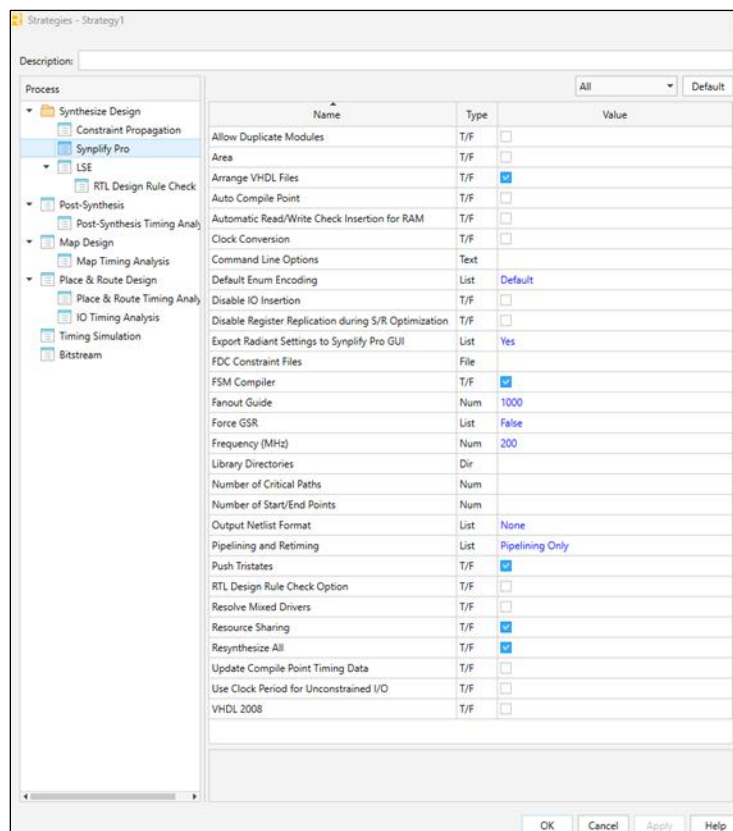


Figure 5.11. Synthesis Strategy Settings

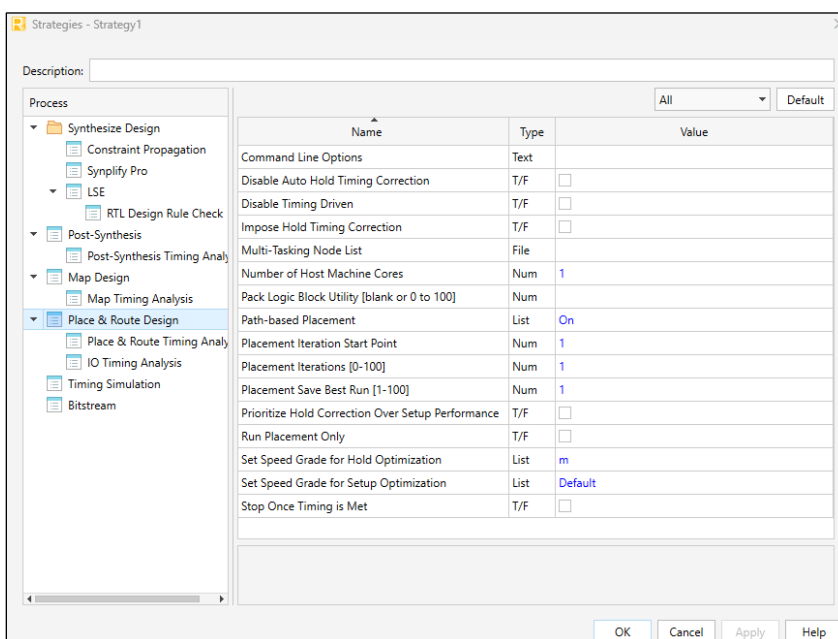


Figure 5.12. Place and Route Design Strategy Settings

10. Click the **Run All** icon to generate the bitstream.

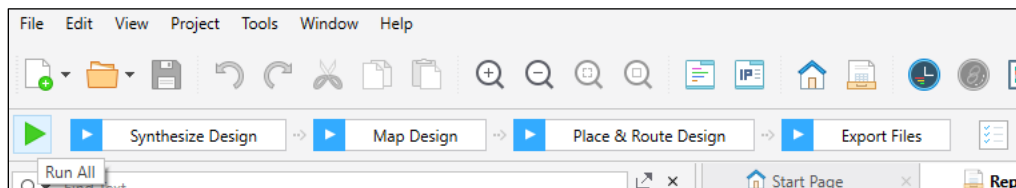


Figure 5.13. Generating Bitstream

11. The compilation flow takes some time to complete. As shown in [Figure 5.14](#), when the compilation completes successfully, the *mipi_csi_to_pcie_impl_1.bit* file is generated or updated in the project path: *<Project Directory>/mipi_csi_to_pcie /impl_1*.



Figure 5.14. Compilation Completed

Note: If timing violations occur during compilation or fail to generate bitstream, refer to the [Debugging](#) section for detailed debugging instructions.

6. Implementing the Reference Design on Board

6.1. Hardware Setup

This section covers the hardware setup and the steps to program the reference design to the serial peripheral interface (SPI) memory of the FPGA board.

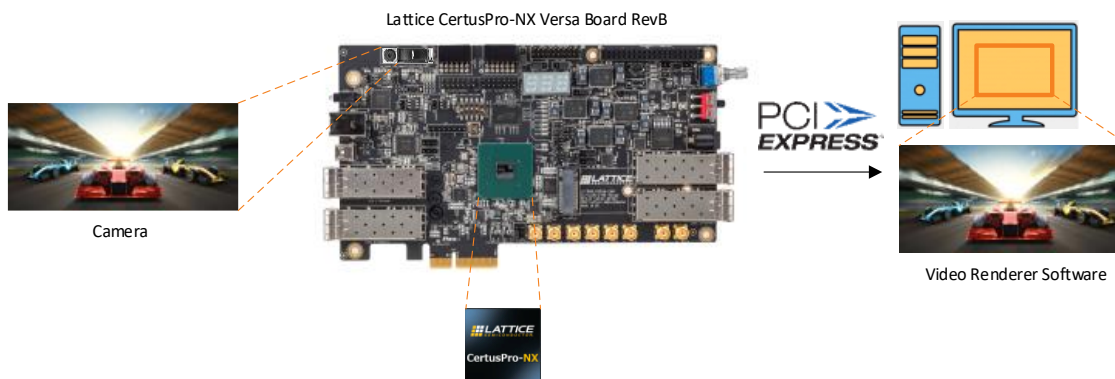


Figure 6.1. MIPI CSI-2 to PCIe Hardware Setup

6.1.1. CertusPro-NX Versa Board RevB

The CertusPro-NX Versa Board has no jumpers to configure out of the default configuration. To ensure PCIe x4 is configured, jumper J1 is shorted at 2–4 by default. The card is plugged into the PCIe slot, external power is provided by the system, and the 12 V source switch receives power from the PCIe slot. The CertusPro-NX Versa Board has an on-board sensor connector, J37, at the top left corner. The Sony IMX258 sensor module must be connected to J37.

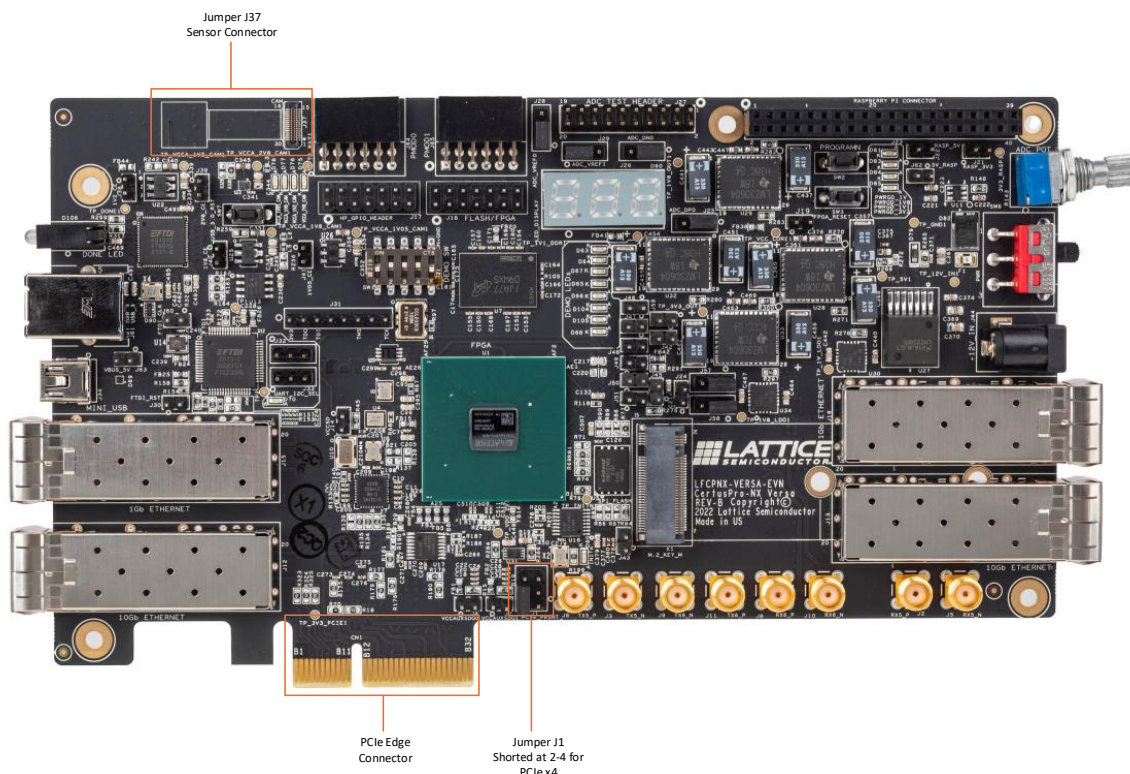


Figure 6.2. CertusPro-NX Versa Board

6.1.2. IMX258 Sensor

The IMX258 sensor module is connected to the CertusPro-NX Versa Board J37.



Figure 6.3. IMX258 Sensor Module

6.1.3. Host PC

The host PC executes the demo software. To set up the system, ensure the PCIe slot in the host PC supports PCIe x4. Insert the CertusPro-NX Versa Board with the IMX258 sensor module into the PCIe slot. Refer to the [Software Setup](#) section for the details of the software setup.

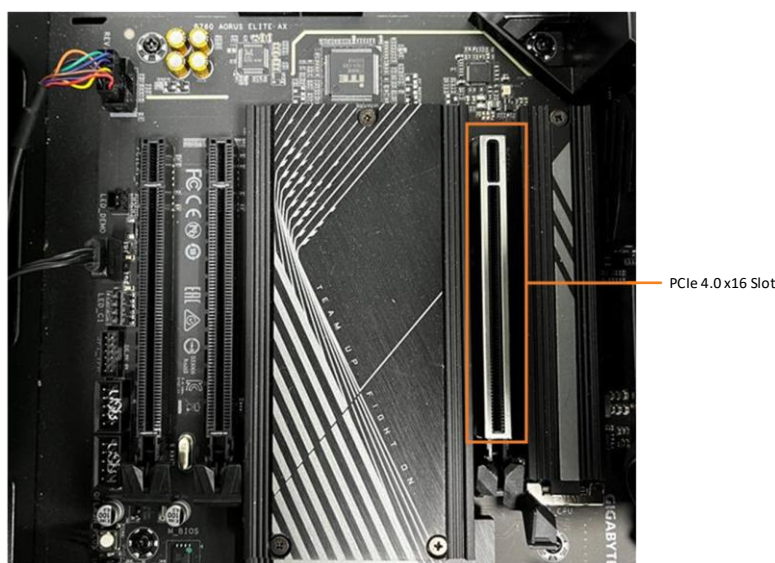


Figure 6.4. Sample PCIe Slot that Supports Up to PCIe x16

6.2. Programming the FPGA

To program the FPGA, ensure the board is inserted into the host PC, then follow these steps:

1. Create a new project using the Lattice Radiant Programmer software. In the **Getting Started** window, input the **Project Name** and **Location** as shown in [Figure 6.5](#).
2. Select **Create a new project from a scan**. The values are indicated in the **Cable**, **Port**, and **TCK Divider Setting (0 30x)** fields.
3. Click **OK**.

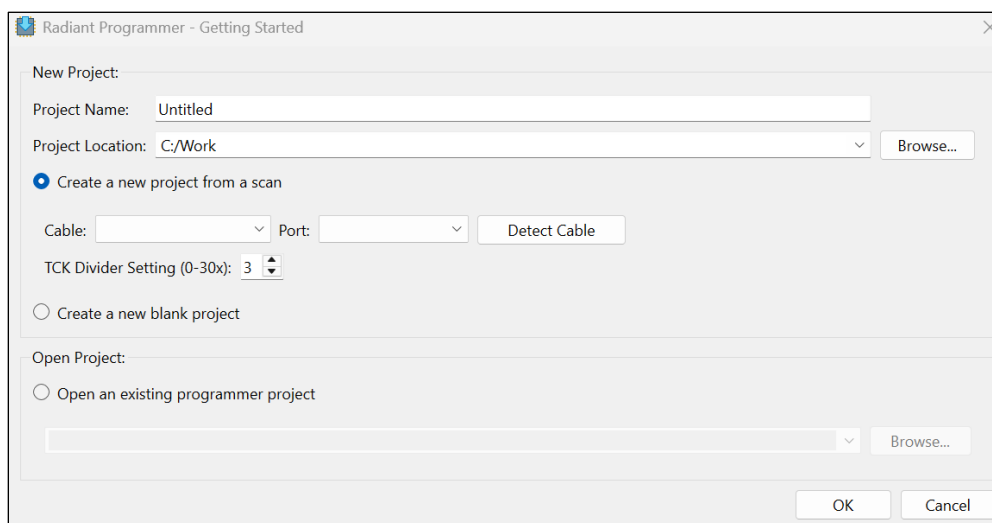


Figure 6.5. Creating a New Project from a Scan

4. The main interface opens as shown in the figure below.

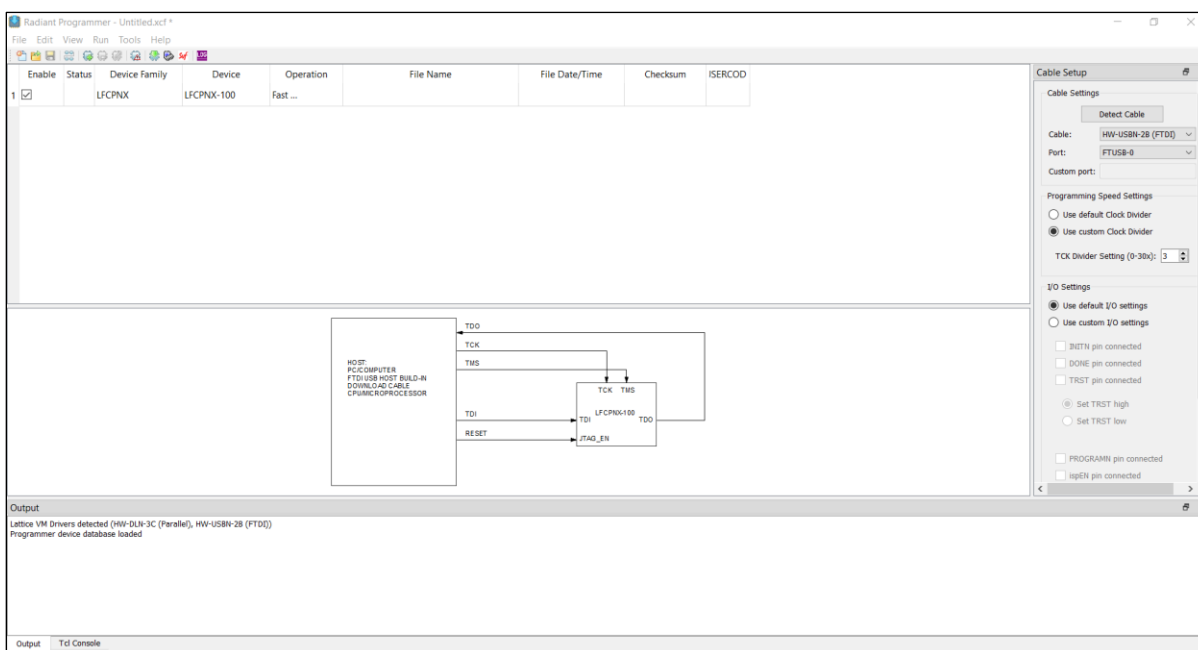


Figure 6.6. Lattice Radiant Programmer Window

5. If the Programmer settings do not match the settings shown in the figure below for CertusPro-NX devices, select these settings manually from the drop-down list.

	Enable	Status	Device Family	Device	Operation	File Name	File Date/Time	Checksum	USERCODE
1	☒		LFCPNX	LFCPNX-100	Fast Configuration				

Figure 6.7. CertusPro-NX Device Settings

To select the programming settings, follow these steps:

1. Browse and select the Programming file, *mipi_csi_to_pcie_impl_1.bit*.
2. Click **OK**.
3. Double-click **Operation** to open the **Device Properties** window.
4. Select the settings as shown in the figure below.

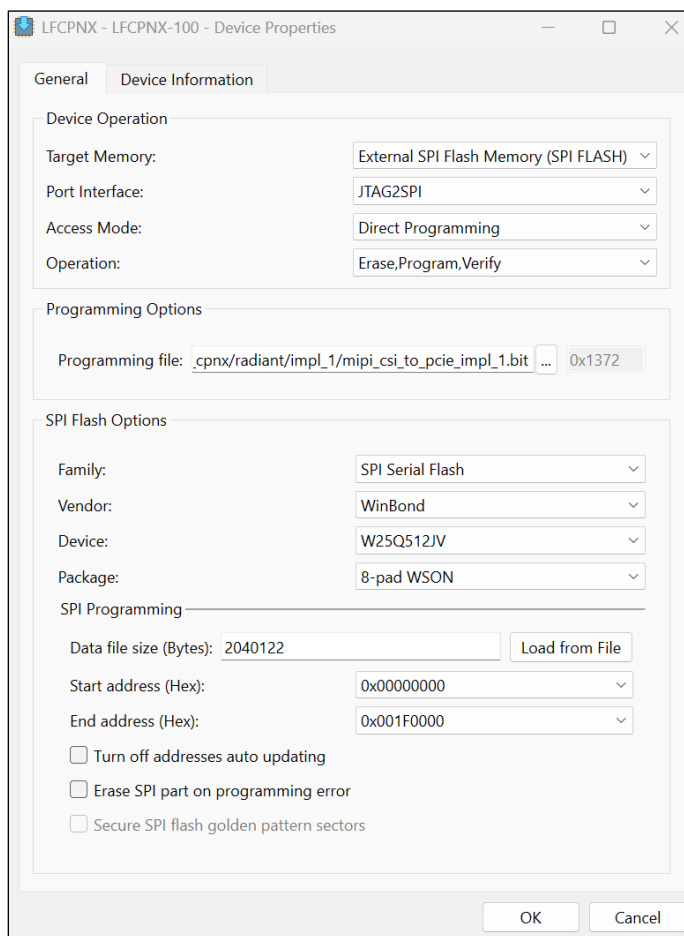


Figure 6.8. Device Properties Window for CertusPro-NX SPI Flash Programming

5. Click **Programming** from the menu bar to start programming.

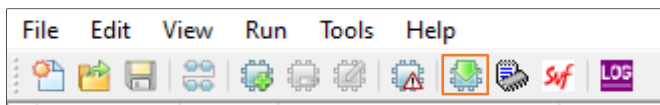


Figure 6.9. Programming Menu Bar

When the FPGA programming is successful, the output console displays the *Operation: successful* message.

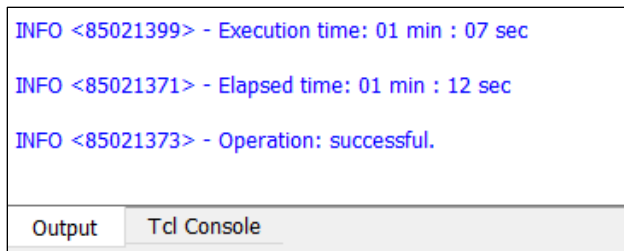


Figure 6.10. Programmer Output Window

If the programming operation is unsuccessful, refer to the [Debugging](#) section.

6. After programming, check the status LEDs on the board, power cycle the board, and reboot the host PC.

6.2.1. Status LED



Figure 6.11. CertusPro-NX Status LEDs

Table 6.1. Description of the Status LED

Sl. No	Name	Description
1	DONE	Green – Lights if configuration is successful.

6.3. Software Setup

This reference design uses the QV4L2 application and the `lsc_pcie_video_bridge` driver to configure the camera by dynamically modifying camera register through the camera control interface (CCI).

6.3.1. QV4L2 Application

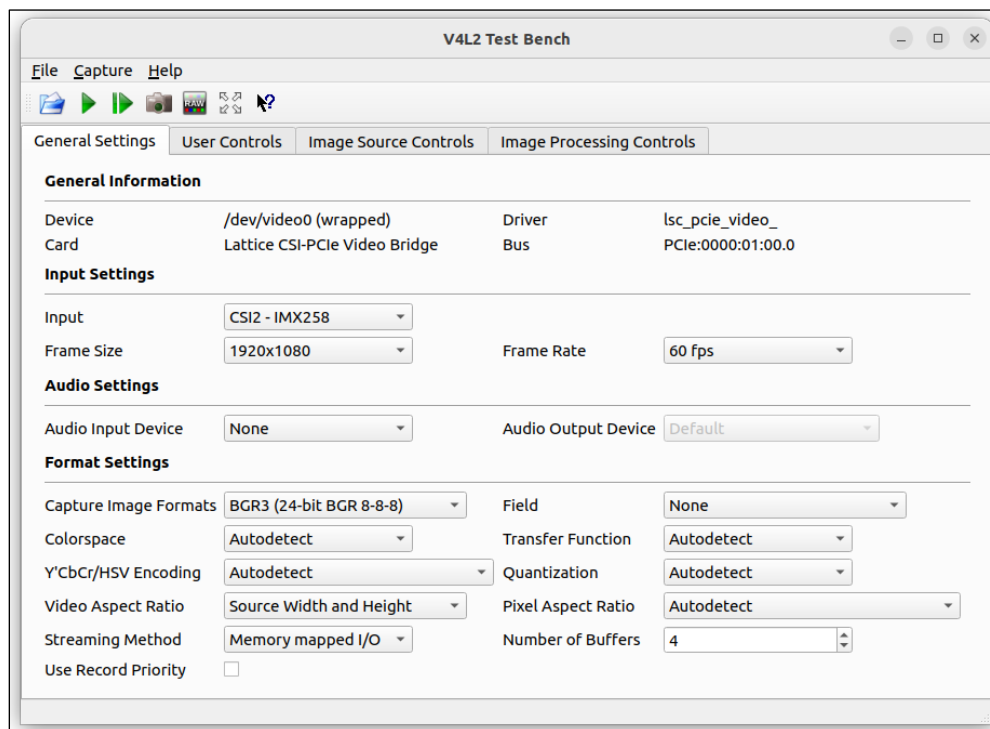


Figure 6.12. QV4L2 Application Window

The QV4L2 application is a third-party open-source graphical test bench application for Linux used to control, test, and view video from Video4Linux2 (V4L2) devices. In this reference design, only the QV4L2 application settings listed in the table below are supported. All other settings are not applicable for this reference design.

Table 6.2. Configurable Settings in the QV4L2 Application

Tab	Settings	Selection
General Settings	Input	CSI-2 – IMX258
	Frame Size	3840x2160 – Only available for MIPI CSI 4 lanes 1920x1080 – Available for MIPI CSI 2/4 lanes 1280x720 – Available for MIPI CSI 2/4 lanes 720x480 – Available for MIPI CSI 2/4 lanes
	Frame Rate	60 fps – Only available for MIPI CSI 4 lanes 30 fps – Available for MIPI CSI 2/4 lanes Note: 3840x2160 only supports 30 fps
	Capture Image Formats	BGR3 (24-bit BGR 8-8-8)
	Number of Buffers	Minimum 2 and can support up to 4
User Controls	Exposure	Slider to adjust camera exposure
	Digital Gain GR/R/B/GB	Slider to adjust camera digital gain
Image Source Controls	Vertical Blanking	Read only
	Horizontal Blanking	Read only
	Analog Gain	Slider to adjust camera analog gain

6.3.2. lsc_pcie_video_bridge Driver

The lsc_pcie_video_bridge driver is a custom Linux out-of-tree kernel driver developed for this reference design. This driver facilitates high-speed video data transfer from the FPGA to the host PC via a scatter-gather DMA engine. The driver implements the standard Video4Linux2 (V4L2) framework, exposing the connected IMX258 image sensor as a standard video capture device (/dev/videoX). This driver allows you to easily capture and view video streams using standard V4L2-compatible applications such as QV4L2, v4l2-ctl, and GStreamer.

The driver supports 2-lane and 4-lane MIPI CSI-2 configurations via the csi_lanes module parameter.

6.3.3. Running the QV4L2 Application and the lsc_pcie_video_bridge Driver

This section provides the steps and instructions for installing drivers and running the demo on a Linux-based host computer. It is recommended to use Ubuntu 22.04.5 LTS 64-bit as the operating system.

The installation script provided automatically compiles the driver for the installed kernel.

Before running the scripts, ensure the following prerequisite packages are installed:

- make
- gcc
- v4l-utils
- qv4l2

Note: On Ubuntu, these packages can be installed using the following command:

```
sudo apt install build-essential v4l-utils qv4l2
```

To install drivers and run the demo on Linux, follow these steps:

1. Insert the board into the PCIe slot of the host PC.

Note: Make sure that Secure Boot is disabled in the BIOS/UEFI before booting the PC.

2. Boot the PC into Linux.

Note: Make sure that LED D68 on the board is not lit up. If this LED is lit, the PCIe link is down. If LED is lit, restart the PC.

3. Open a terminal and navigate to the /RD02338_mipi_csi_to_pcie/misc/software/lsc_pcie_video_bridge folder that you have cloned.

4. Make the scripts executable by running the following command:

```
sudo chmod +x scripts/*.sh
```

5. Verify the Secure Boot status on Ubuntu by running the following command:

```
$ sudo mokutil --sb-state
```

This command outputs whether Secure Boot is enabled or disabled. Secure Boot must be disabled because Secure Boot prevents the loading of out-of-tree kernel modules such as this demo driver. If Secure Boot is enabled, you must disable Secure Boot in your BIOS/UEFI settings.

6. Run the demo script that corresponds to your hardware configuration. Run only one of the following commands:

- For MIPI CSI 4-lane configuration:

```
sudo ./scripts/run_demo_4lanes.sh
```
- For MIPI CSI 2-lane configuration:

```
sudo ./scripts/run_demo_2lanes.sh
```

These scripts automatically compile the .ko driver file (if the driver has not been built yet), load the required kernel dependencies, and insert the driver. When the driver loads successfully, the scripts automatically open the QV4L2 application window as shown in [Figure 6.12](#).

7. In the QV4L2 application, select your desired video settings from the **Frame Size** and **Frame Rate** drop-down list. Only supported resolutions and frame rates are displayed.

8. Click the play button  to start streaming. A new window opens to display the live rendered video stream as shown in [Figure 6.13](#).

Notes:

- The driver is not loaded automatically on boot. Navigate to the lsc_pcie_video_bridge folder and rerun step 6 every time the PC is restarted.

- Restart the system if an issue is encountered when executing commands in step 6.
- If the **Frame Size** drop-down list in the QV4L2 application is empty, reselect **BGR3** for **Capture Image Formats**. This value populates the **Frame Size** drop-down list.
- Optionally, if you want to use a dedicated V4L2 compatible application, replace step 6, 7, and 8 with following steps:
 1. Compile the driver if the driver has not been built yet:

```
sudo ./scripts/compile.sh
```
 2. Install the driver that corresponds to your configuration. Run only one of the following commands:
 - For MIPI CSI 4-lane configuration:

```
sudo ./scripts/install_drv_4lanes.sh
```
 - For MIPI CSI 2-lane configuration:

```
sudo ./scripts/install_drv_2lanes.sh
```
 3. Run your V4L2 compatible application and configure the settings to the supported frame size, frame rate, and pixel format (**BGR3**).
 4. To remove the driver when you are finished, run the following command:

```
sudo ./scripts/rm_drv.sh
```

Figure 6.13 shows the rendered streaming video data that is captured by the Sony IMX258 sensor on screen.

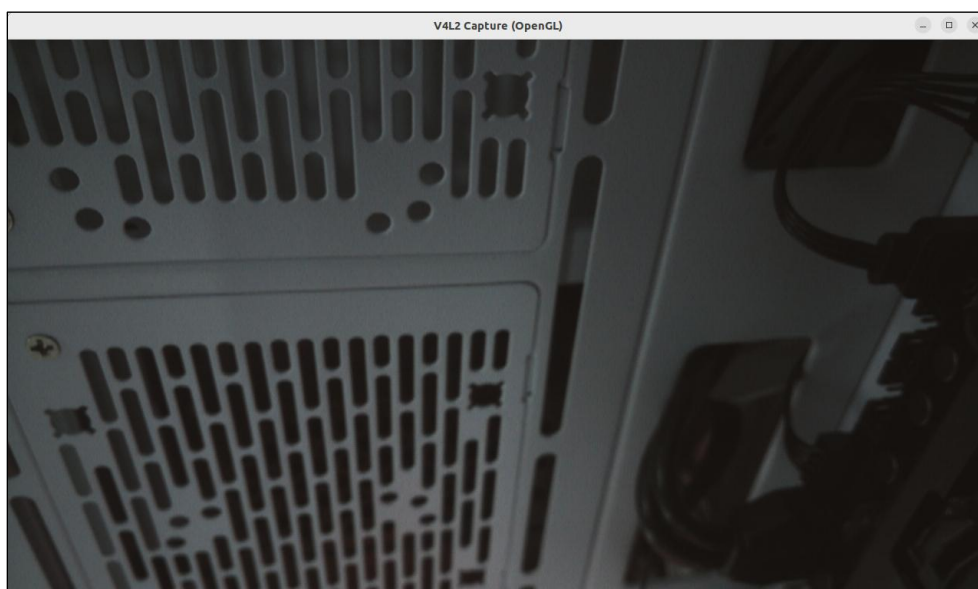


Figure 6.13. MIPI CSI-2 to PCIe Bridge Output Image Using the QV4L2 Application

7. Customizing the Reference Design

This section describes the steps required to configure each IP and software for various demonstration setups, starting from the default design configuration.

While the default IP configuration of 4 lanes with 4 pixels per clock can support all resolutions and frame rates, the IP can be reconfigured to demonstrate lower lane count and pixel per clock setup resulting in reduced I/O and resource utilization. The IMX258 sensor is configured to operate at 1200 Mbps link rate with either 2 or 4 lanes configuration. Refer to [Table 7.1](#) for the IP configuration settings for the desired setup validated on the CertusPro-NX Versa Board.

Table 7.1. IP Configuration for Supported Resolutions

Resolution	D-PHY Link Rate (Mbps) ¹	D-PHY Lane Count ²	Pixels Per Clock Required ³ (Clocking at 150 MHz)
4K30	1200	4	4
1080p60	1200	4	4
1080p30	1200	4	4
1080p30	1200	2	2 or 4
720p60	1200	4	4
720p30	1200	4	4
720p30	1200	2	2 or 4
480p60	1200	4	4
480p30	1200	4	4
480p30	1200	2	2 or 4

Notes:

1. The IMX258 sensor does not support link rate higher than 1200 Mbps.
2. The IMX258 sensor does not support 1 lane configuration.
3. Pixels per clock required is determined by calculating the clocking ratio with the following formula. The clocking required is clocking ratio multiplied by byte clock frequency.

$$(\text{NUM_LANES} \times \text{PHY_DATA_WIDTH}) / (\text{PPC} \times \text{BPP})$$

where in this reference design, **NUM_LANES** is 4 or 2, **PHY_DATA_WIDTH** is 8, **PPC** is 1, 2, or 4, **BPP** is 10 (RAW10) with minimum ratio 1, byte clock frequency is 1200 Mbps / 8 = 150 MHz.

- **NUM_LANES 4, PPC 1:** $(4 \times 8) / (1 \times 10) = 3.2 \times \text{clocking ratio}$; clocking required: $3.2 \times 150 \text{ MHz} = 480 \text{ MHz}$
- **NUM_LANES 4, PPC 2:** $(4 \times 8) / (2 \times 10) = 1.6 \times \text{clocking ratio}$; clocking required: $1.6 \times 150 \text{ MHz} = 240 \text{ MHz}$
- **NUM_LANES 4, PPC 4:** $(4 \times 8) / (4 \times 10) = 0.8 \times \text{clocking ratio}$; clocking required: $1 \times 150 \text{ MHz} = 150 \text{ MHz}$
- **NUM_LANES 2, PPC 1:** $(2 \times 8) / (1 \times 10) = 1.6 \times \text{clocking ratio}$; clocking required: $1.6 \times 150 \text{ MHz} = 240 \text{ MHz}$
- **NUM_LANES 2, PPC 2:** $(2 \times 8) / (2 \times 10) = 0.8 \times \text{clocking ratio}$; clocking required: $1 \times 150 \text{ MHz} = 150 \text{ MHz}$
- **NUM_LANES 2, PPC 4:** $(2 \times 8) / (4 \times 10) = 0.4 \times \text{clocking ratio}$; clocking required: $1 \times 150 \text{ MHz} = 150 \text{ MHz}$

For more information, refer to the <Project Directory>/latticesemi.com_ip_mipi_csi_dsi_1.0.0.01/doc/IP_Design_Notes.pdf file for the IP user guide.

7.1. Changing IP Configuration to 2 Lanes, 4 PPC

This section describes the steps to configure IP to support 2 lanes, 4 PPC.

No configuration is needed for the following IPs as the default configuration is 4 PPC:

- Debayer IP
- Color Correction Matrix IP
- Automatic White Balance IP
- Video to PCIe Bridge

7.1.1. Configuring MIPI RX IP

To configure the MIPI RX IP, follow these steps:

1. Double-click the mipi_csi_rx_inst block to open the Module/IP Block Wizard.
2. Under **PHY Configuration**, set **Number of D-PHY RX Lanes** to 2.

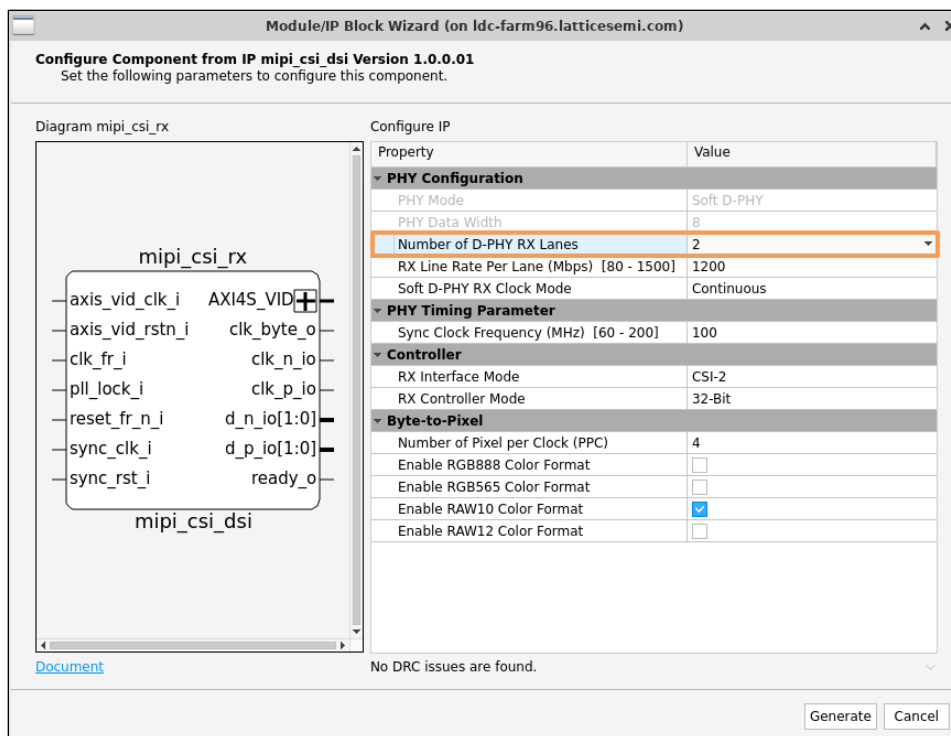


Figure 7.1. MIPI RX IP Configuration

3. Click **Generate**.
4. After generation is complete, click **Finish** in the **Check Generated Result** window.

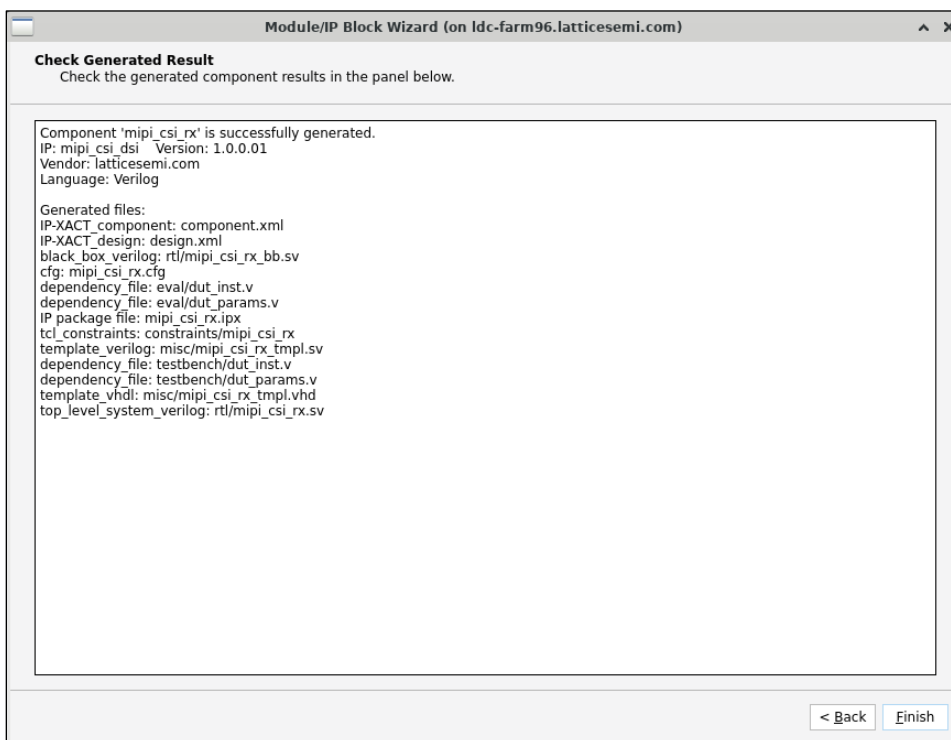


Figure 7.2. MIPI RX IP Check Generated Result Window

7.2. Changing IP Configuration to 2 Lanes, 2 PPC

This section describes the steps to configure IP to support 2 lanes, 2 PPC.

7.2.1. Configuring MIPI RX IP

To configure the MIPI RX IP, follow these steps:

1. Double-click the `mipi_csi_rx_inst` block to open the Module/IP Block Wizard.
2. Under **PHY Configuration** and **Byte-to-Pixel**, set **Number of D-PHY RX Lanes** to 2 and **Number of Pixel per Clock (PPC)** to 2.

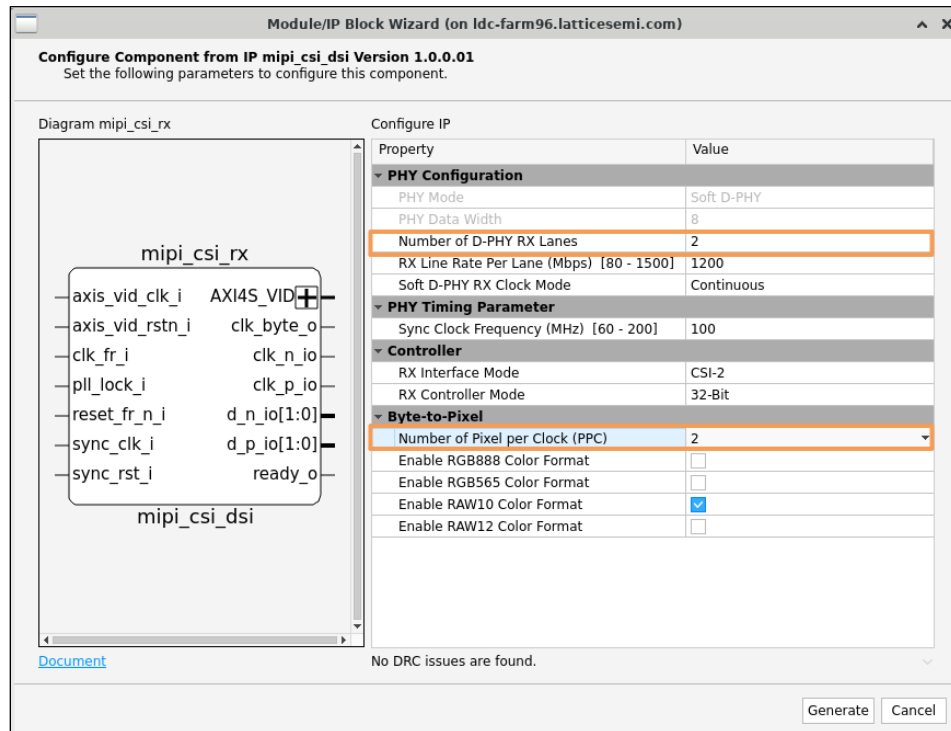


Figure 7.3. MIPI RX IP Configuration

3. Click **Generate**.
4. After generation is complete, click **Finish** in the **Check Generated Result** window.

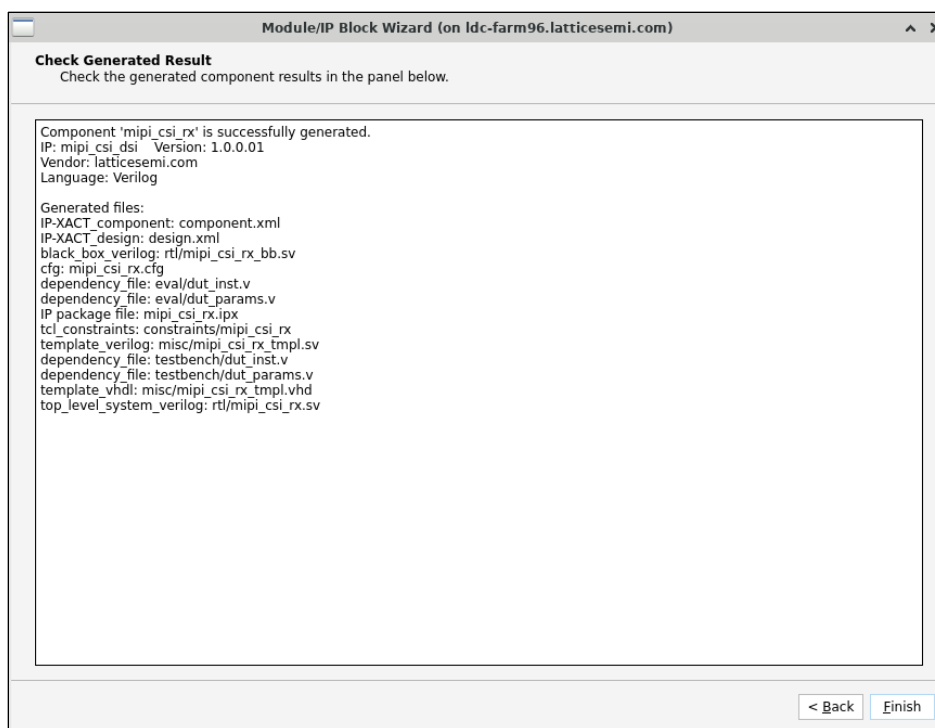


Figure 7.4. MIPI RX IP Check Generated Result Window

7.2.2. Configuring Debayer IP

To configure the Debayer IP, follow these steps:

1. Double-click the isp_debayer_inst block to open the Module/IP Block Wizard.
2. Under **Debayer Configuration**, set **Pixels per clock** to 2.

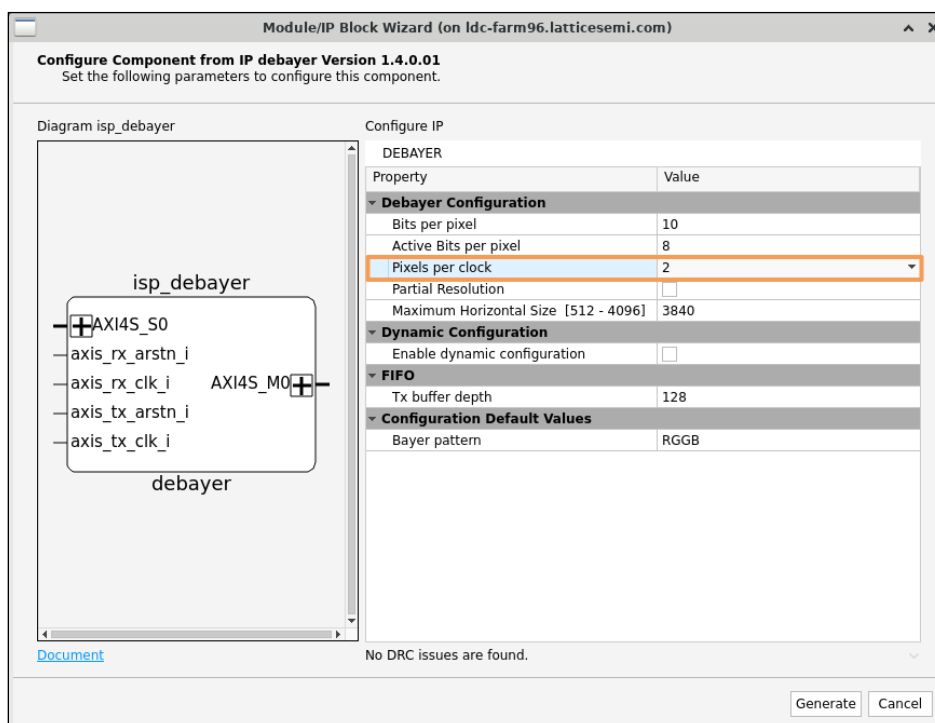


Figure 7.5. Debayer IP Configuration

3. Click **Generate**.
4. After generation is complete, click **Finish** in the **Check Generated Result** window.

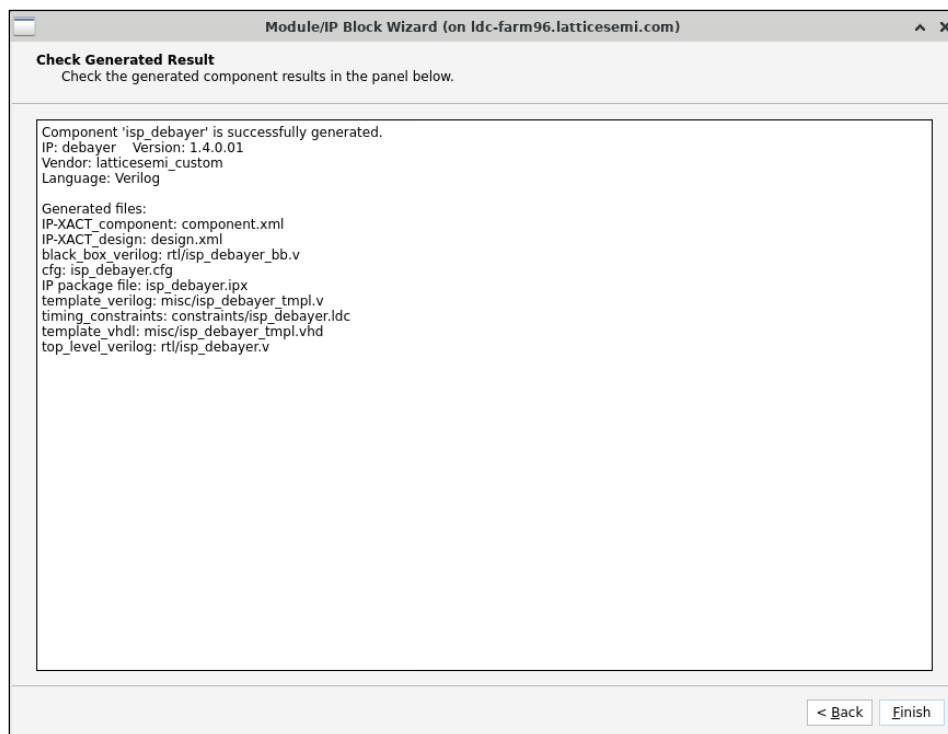


Figure 7.6. Debayer IP Check Generated Result Window

7.2.3. Configuring Color Correction Matrix IP

To configure the Color Correction Matrix IP, follow these steps:

1. Double-click the isp_ccm_inst block to open the Module/IP Block Wizard.
2. Under **CCM Configuration**, set **Pixels per clock** to 2.

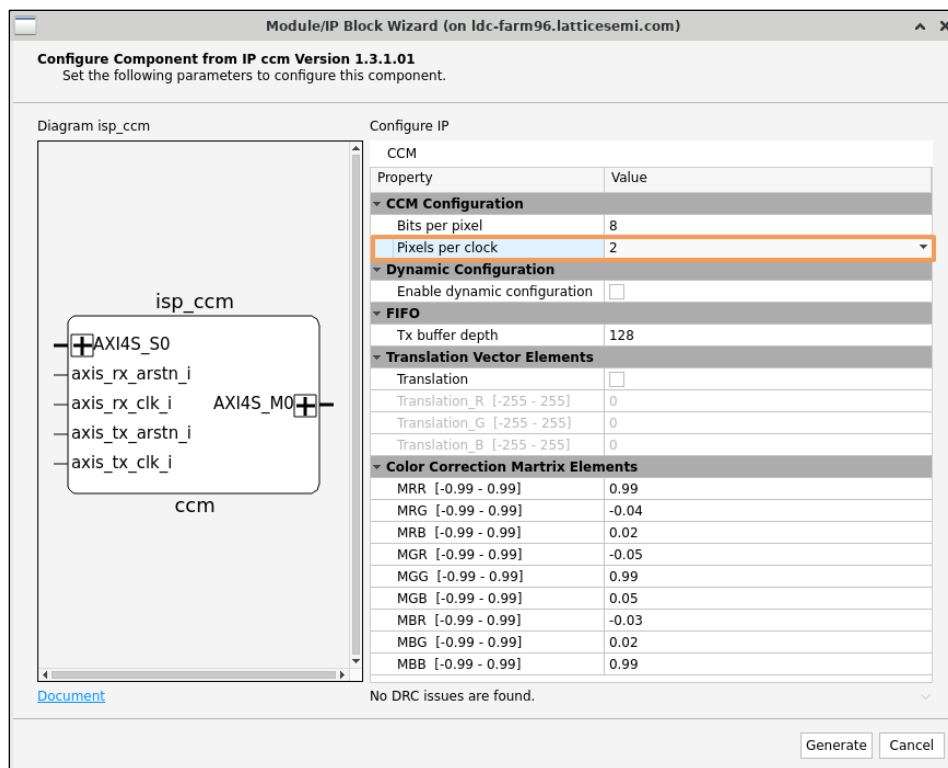


Figure 7.7. Color Correction Matrix IP Configuration

3. Click **Generate**.
4. After generation is complete, click **Finish** in the **Check Generated Result** window.

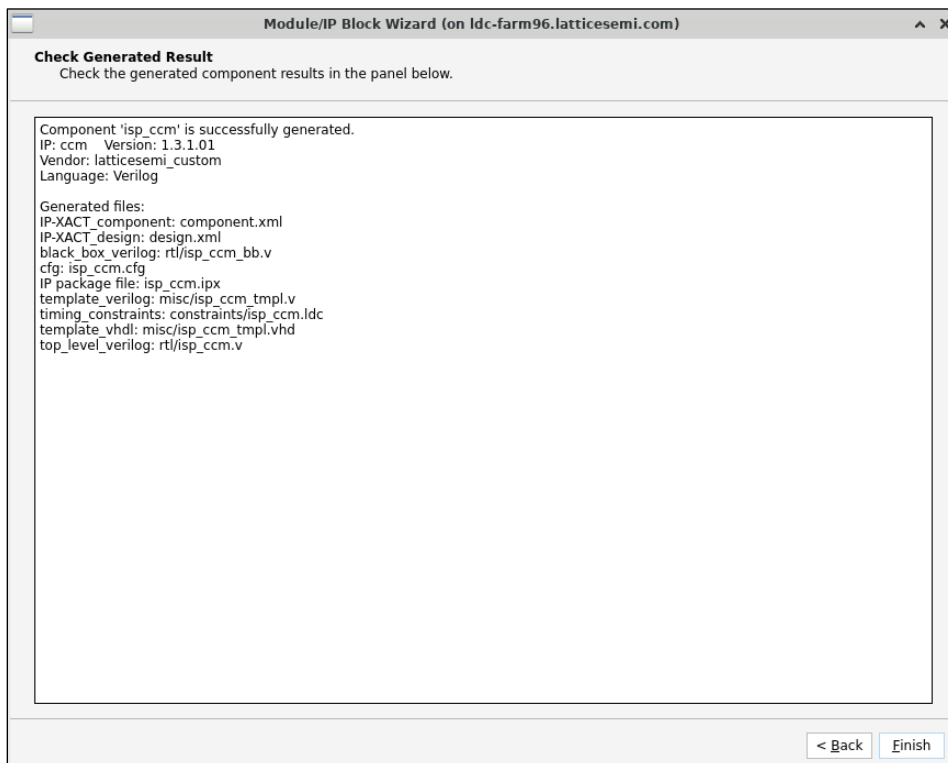


Figure 7.8. Color Correction Matrix IP Check Generated Result Window

7.2.4. Configuring Automatic White Balance IP

To configure the Automatic White Balance IP, follow these steps:

1. Double-click the isp_awb_inst block to open the Module/IP Block Wizard.
2. Under **AWB Configuration**, set **Pixels per clock** to 2.

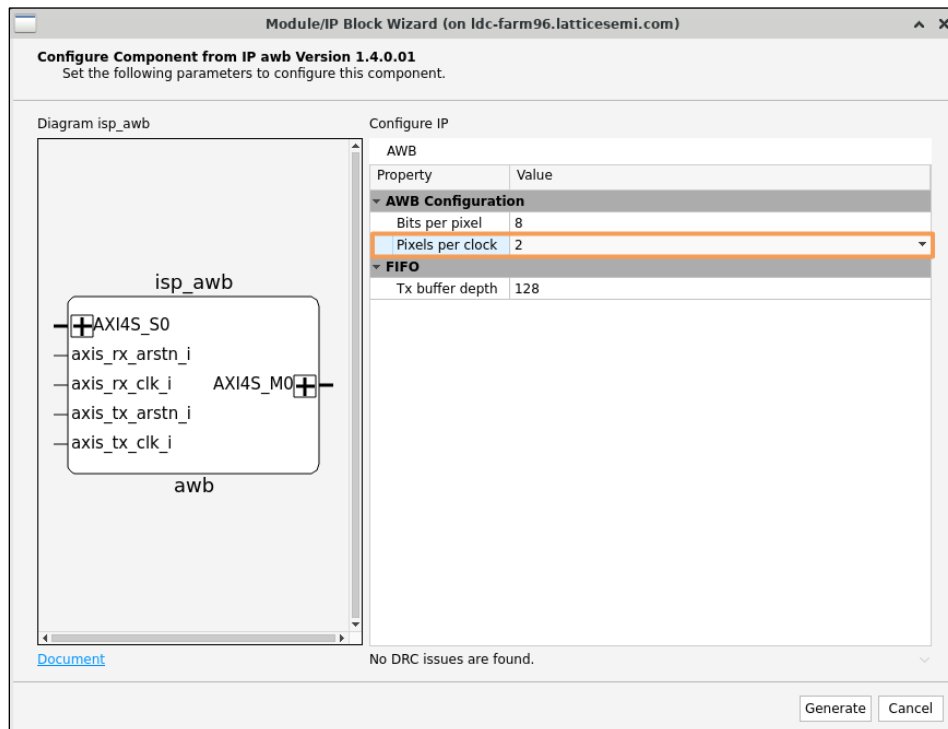


Figure 7.9. Automatic White Balance IP Configuration

3. Click **Generate**.
4. After generation is complete, click **Finish** in the **Check Generated Result** window.

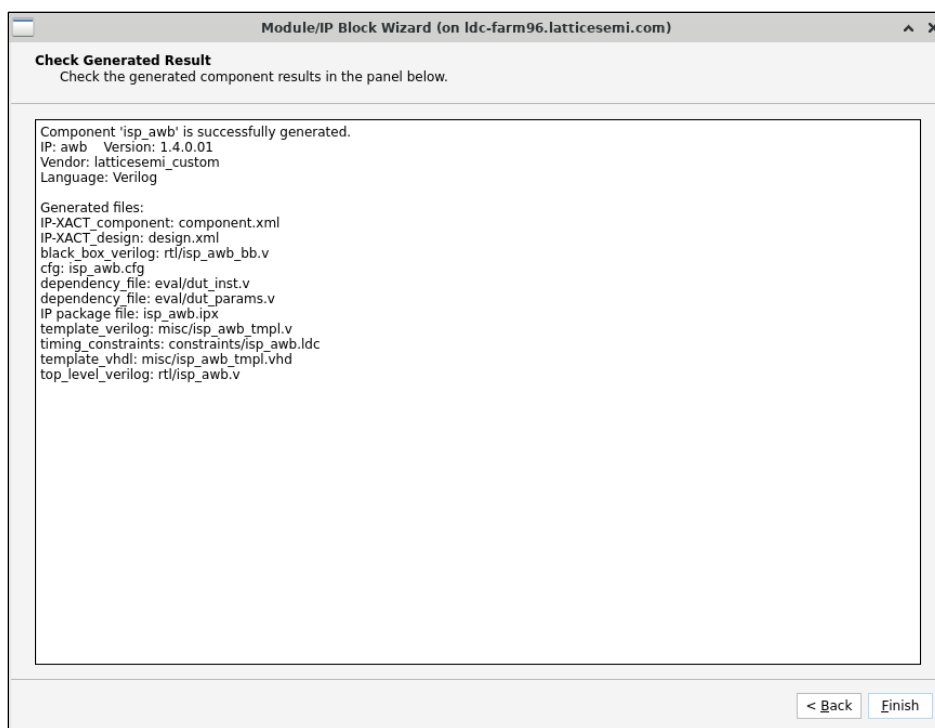


Figure 7.10. Automatic White Balance IP Check Generated Result Window

7.2.5. Configuring Video to PCIe Bridge

To configure the Video to PCIe Bridge, follow these steps:

1. Double-click the video_pcie_bridge_inst block to open the Module/IP Block Wizard.
2. Set **Pixels Per Clock (PPC)** to 2.

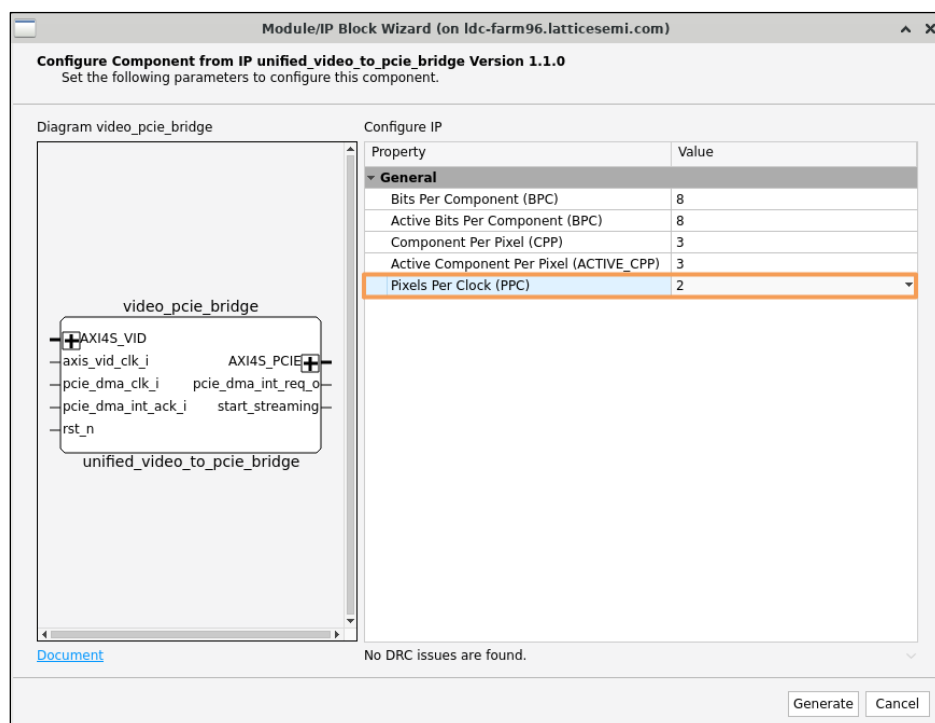


Figure 7.11. Video to PCIe Bridge Configuration

3. Click **Generate**.
4. After generation is complete, click **Finish** in the **Check Generated Result** window.

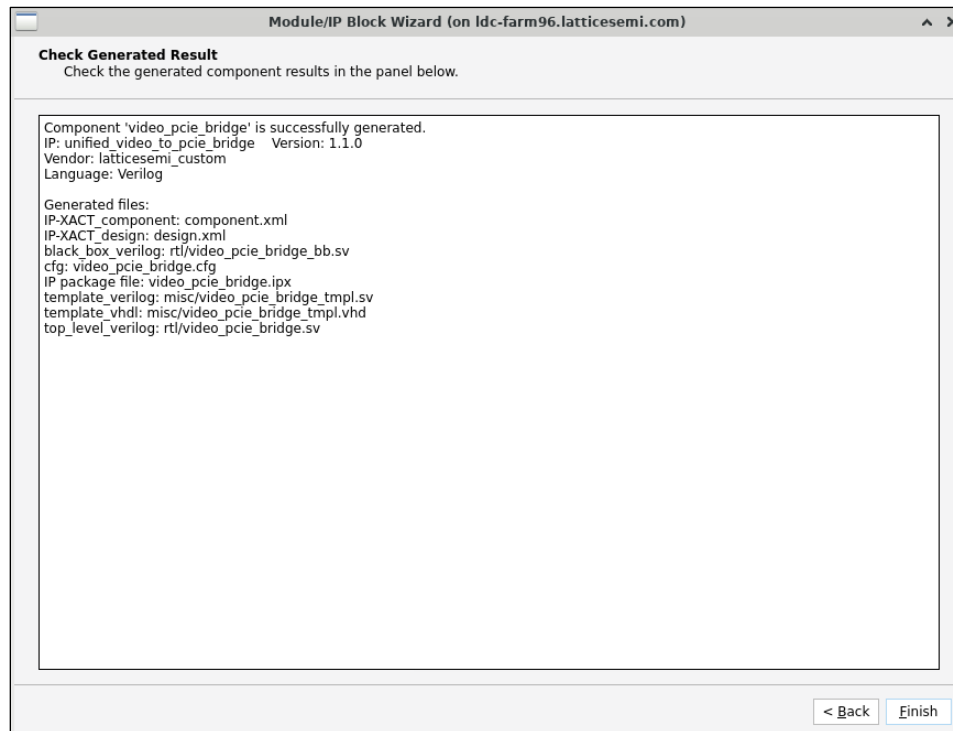


Figure 7.12. Video to PCIe Bridge Check Generated Result Window

After reconfiguration is complete, refer to the [Running the Reference Design](#) section to generate the bitstream. After bitstream is generated, refer to the [Programming the FPGA](#) section to program the bitstream into the FPGA.

8. Resource Utilization

The table below shows the example configurations validated on the CertusPro-NX Versa Board, including the resource utilization for the MIPI CSI-2 to PCIe reference design. These configurations use the LFCPNX-100-9LFG672C device.

Table 8.1. Resource Utilization of the CertusPro-NX MIPI CSI-2 to PCIe Reference Design

MIPI CSI/DSI RX		ISP (Debayer, CCM, AWB)		Resource Utilization			
Lane Count	Line Rate (Mbps)	BPC	PPC	LUT4	Registers	DSP	EBR
x4	1200	8	4	28863	21831	68	53
x2	1200	8	4	28436	21659	68	51
x2	1200	8	2	21982	17224	38	50

9. Debugging

9.1. Error when Updating SPI Flash

- If there is verification error when programming the .bit file, try changing the TCK frequency by setting **TCK Divider Setting** to greater than 3. Click **Program** to restart programming.

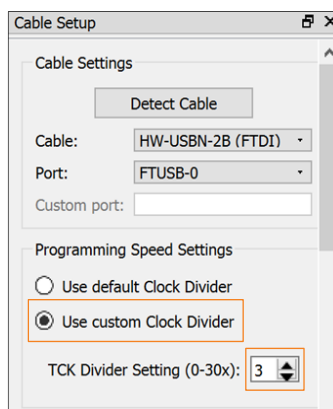


Figure 9.1. TCK Frequency Settings

If the verification error problem still occurs, press and hold the **PROGRAMN** push button before clicking **Program**.

- If the device is not detected on Port: **FTUSB-0**, click **Detect Cable** and select Port: **FTUSB-1**.

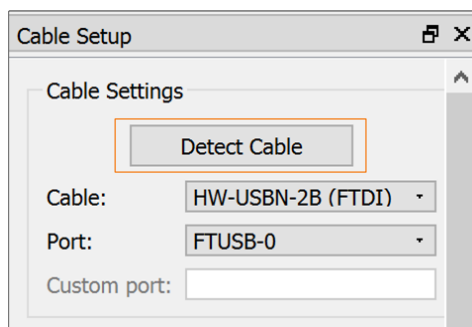


Figure 9.2. Port Selection

9.2. Timing Violations

Timing violations may occur during compilation and may be caused by the Placement Iteration settings for Place and Route not set optimally on the current machine. To resolve this, follow these steps:

- In the Lattice Radiant software, go to **Project > Active Strategy > Place & Route Design Settings**.
- In the settings window as shown in Figure 9.3, change both **Placement Iteration** and **Placement Save Best Run** from **1** to **10**. Select **Stop Once Timing is Met**.

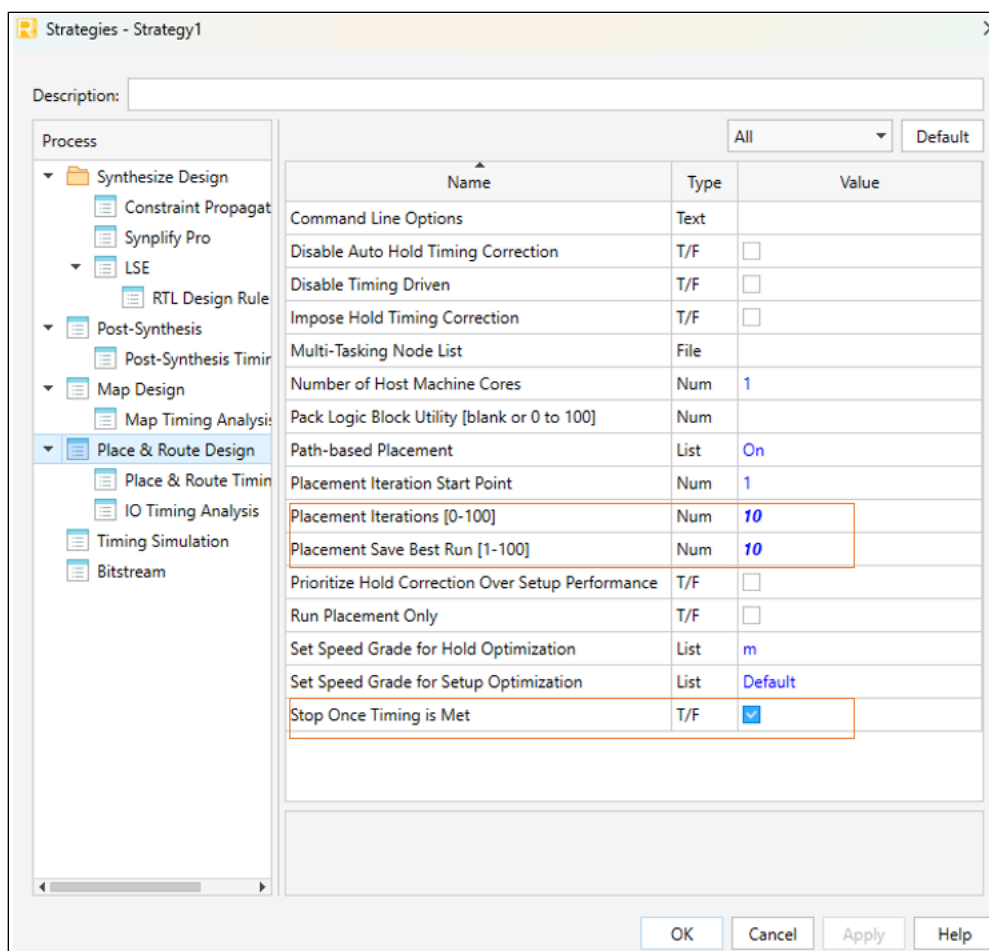


Figure 9.3. Place & Route Design Strategy Settings

3. Click **OK** to apply the changes.
4. Click **Export Files** to regenerate the bitstream.

If an iteration meets timing requirements, the Place and Route reports display the placement results.

References

- [Lattice Radiant Timing Constraints Methodology \(FPGA-AN-02059\)](#)
- [CertusPro-NX](#) web page
- [CertusPro-NX Versa Board](#) web page
- [PCI Express for Nexus FPGAs](#) web page
- [MIPI CSI-2 to PCIe Reference Design](#) web page for design files from GitHub
- [Lattice Propel Builder Software](#) web page
- [Lattice Radiant Software](#) web page
- [Lattice Insights](#) for Lattice Semiconductor training courses and learning plans

Technical Support Assistance

Submit a technical support case through www.latticesemi.com/techsupport.

For frequently asked questions, refer to the Lattice Answer Database at www.latticesemi.com/Support/AnswerDatabase.

Revision History

Revision 1.0, April 2026

Section	Change Summary
All	Initial release.



www.latticesemi.com